

Aplicación de modelos de visión artificial para el conteo de plantas en cultivos de arroz

Prof. Alejandro Manuel Lloveras

Carrera de Especialización en Inteligencia Artificial

Director: Ing. Juan Ignacio Cavalieri (FIUBA)

Codirector: Dr. Marcel Bentancor (UdelaR)

Jurados:

Dr. Ing. Facundo Luciana (UNT/FIUBA)

Esp. Ing. Alfonso Rafel (UNR/FIUBA)

Ing. Martín Horn (UNLu/FIUBA)

Ciudad Autónoma de Buenos Aires, junio de 2025

Resumen

La presente memoria describe el desarrollo de un *pipeline* de visión artificial, desarrollado en colaboración con la empresa Adecoagro. El objetivo es automatizar el conteo de plantas en cultivos de arroz para mejorar la eficiencia y reducir costos en el monitoreo.

Para realizar el trabajo se emplearon conocimientos de análisis de datos, estadística y análisis espacial, gestión de archivos, procesamiento de imágenes, índices de vegetación, *data augmentation*, visión artificial con aprendizaje supervisado y no supervisado, aprendizaje profundo (arquitecturas CNN), *transfer learning* y optimización de hiperparámetros.

Agradecimientos

Agradezco en primer lugar a los directores, quienes me han acompañado en esta ardua faena. También me gustaría dar las gracias a los jurados por haberse interesado en mi tesis y el tiempo dedicado a su corrección.

Así mismo, quisiera agradecerle a los revisores de la Facultad de Ingeniería de la UBA, cuya exhaustiva corrección de estilo de cada capítulo y valiosos consejos fueron de gran utilidad durante la redacción de esta memoria.

Por último, pero no menos importante, quiero agradecer a mis padres, que siempre han promovido la curiosidad y el deseo por conocer.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción al muestreo de cultivos	1
1.2. Motivación	1
1.3. Estado del arte	2
1.4. Objetivos y alcance	4
2. Introducción específica	5
2.1. Vehículos aéreos no tripulados (UAV)	5
2.1.1. Características de los drones utilizados	5
2.2. Tratamiento de datos	6
2.2.1. Etiquetado de imágenes	6
2.2.2. Técnica de aumento de datos (<i>Data Augmentation</i>)	6
2.3. Modelos de inteligencia artificial utilizados	7
2.3.1. Arquitecturas convolucionales (CNN)	7
2.3.2. Arquitectura YOLO	7
2.3.3. Técnica de <i>Fine-Tuning</i> (<i>Transfer Learning</i>)	8
2.4. Algoritmo DBSCAN	8
2.5. Índices de vegetación: <i>Excess Green</i>	9
2.6. Análisis de componentes principales (PCA)	10
2.7. Ecualización adaptativa del histograma (CLAHE)	10
3. Diseño e implementación	11
3.1. Arquitectura del sistema	11
3.2. Captura de imágenes	11
3.2.1. Características del cultivo	12
3.2.2. Plan de vuelo de drones	12
3.3. Construcción del dataset	14
3.3.1. Clases y estrategia de etiquetado	15
Gestión de nombres de archivos	15
3.3.2. Aumentado de datos	16
Estrategia de <i>Data Augmentation</i> de 1 etapa	16
Aplicación para el dataset con falso color	16
Estrategia de <i>Data Augmentation</i> de 2 etapas	16
Estrategia de <i>Data Augmentation</i> suave	17
3.3.3. Aplicación de índices de vegetación (dataset de falso color)	17
<i>Excess Green</i> (ExG)	17
Reducción dimensional (2-PCA)	17
Falso color (FC)	19
Subexposición de color (<i>color burn</i>)	20
3.3.4. Imágenes de suelo	20

3.4.	Preprocesamiento de imágenes y etiquetas	21
3.4.1.	División en mosaicos	21
	<i>Offset</i> (opcional)	21
	División de imágenes	22
	División de etiquetas (solo para entrenamiento)	22
	Gestión de nombres de archivos	23
3.5.	Ecualización adaptativa (CLAHE)	23
3.6.	Normalización (escalado de rango)	24
3.7.	Preprocesamiento para falso color (opcional)	24
3.8.	Modelo de detección de objetos	24
3.8.1.	<i>Split</i> de datos	24
3.8.2.	Entrenamiento de modelos	25
3.8.3.	Validación de modelos	26
3.8.4.	Inferencia: módulo de detección de plantas	27
3.9.	Posprocesamiento de predicciones	27
3.9.1.	Reconstrucción y limpieza de etiquetas	28
3.9.2.	Detección de líneas de cultivo	28
3.9.3.	Discriminación de malezas	30
3.9.4.	Conteo de plantas y cálculo de densidad	30
3.10.	Procesamiento por lote	31
3.11.	Batería de métricas	31
4.	Ensayos y resultados	33
4.1.	Ensayos con modelos	33
4.1.1.	Pruebas con diferentes arquitecturas	34
4.2.	Construcción del Dataset	34
4.2.1.	Resultados con imágenes anguladas	35
4.2.2.	Resultados para distintas alturas de vuelo	36
4.3.	Configuración de YOLOv8	36
4.3.1.	<i>Transfer learning</i>	36
4.3.2.	Ajuste de hiperparámetros	37
	Función de pérdida ponderada (<i>Loss function</i>)	37
	Entrenamiento multiescala (<i>multi_scale</i>)	37
	Optimizador SGD	37
	<i>Momentum</i>	38
	Regularización L2 (<i>weight_decay</i>)	38
	<i>Dropout</i>	38
	Otros hiperparámetros explorados	38
4.4.	Desempeño de los mejores modelos	39
4.5.	Validación de modelos	40
4.5.1.	Impacto de IoU y confianza	41
4.6.	Testeo de modelos	41
4.6.1.	Impacto del desarrollo de la planta en la detección	42
4.7.	Desempeño en producción	44
4.7.1.	Caracterización del sistema productivo	45
	Crítica del método tradicional	45
4.7.2.	Impacto del desarrollo de la planta en la <i>performance</i>	46
4.7.3.	Reflexión sobre el umbral de confianza y IoU	46
4.7.4.	Casos problemáticos	47
4.7.5.	Impacto de la altura de vuelo en la <i>performance</i>	47
4.8.	Estimación de recursos necesarios	47

4.8.1. Aporte a la productividad	49
5. Conclusiones	51
5.1. Conclusiones generales	51
5.2. Próximos pasos	52
A. Resultados de inferencia	53
B. Número de muestras y precisión en la medición	55
B.1. Muestreo por conglomerados (<i>Cluster sampling</i>)	55
B.2. Impacto del tamaño muestral en el <i>Accuracy</i>	60
C. Protocolo de vuelo de dron	63
D. Recursos de hardware	65
Bibliografía	67

Índice de figuras

2.1. Ejemplo de aumentación de datos ¹	6
2.2. Estructura característica de una red CNN.	7
2.3. Comparación entre algoritmos DBSCAN y <i>K-means</i> ²	8
2.4. Ejemplo del proceso de búsqueda de puntos vecinos y detección de <i>outliers</i> ³	9
3.1. Diagrama del <i>pipeline</i> de inferencia.	11
3.2. Error en el conteo de plantas en función de la altura.	13
3.3. Error de conteo por lote en función de la altura.	13
3.4. Diagrama de <i>scripts</i> para el desarrollo del dataset.	14
3.5. Resultado de la transformación <i>Excess Green</i>	18
3.6. Histograma promedio por canal para el dataset.	18
3.7. Resultado de la reducción dimensional con PCA.	19
3.8. Resultado de la fusión <i>Color Burn</i>	19
3.9. Procesamiento de imágenes del suelo.	20
3.10. Imagen original y modificada con <i>offset</i>	21
3.11. Proceso de generación de imágenes de falso color.	24
3.12. Diagrama de <i>scripts</i> para experimentación.	25
3.13. Diagrama de <i>scripts</i> para validación y testeo.	26
3.14. Asignación de <i>clusters</i> realizada por DBSCAN.	29
3.15. Detección de líneas de cultivo.	29
3.16. Detección de malezas (<i>outliers</i>).	30
4.1. Correlograma de etiquetas.	35
4.2. Curvas de F1-score en función de la confianza.	39
4.3. Matriz de confusión para el conjunto de validación.	40
4.4. Curvas de <i>Precision</i> en función de la confianza y <i>recall</i>	41
4.5. Matriz de confusión para el conjunto de testeo.	42
4.6. Matriz de confusión para planta con desarrollo avanzado.	43
4.7. Ejemeplo de inferencia para lote con anegamiento.	44
4.8. Impacto del IoU y <i>Confidence</i> en la inferencia.	46

Índice de tablas

3.1. Datos de Campos	12
3.2. Primera captura de imágenes (2/10/2024).	12
3.3. Segunda captura de imágenes (12/12/2024).	14
3.4. Transformaciones de Roboflow.	16
3.5. Transformaciones de Albumentations (YOLO <i>default</i>).	17
3.6. Valores del área cubierta por la fotografía.	31
4.1. Distribución de etiquetas por clase.	35
4.2. Resultados para <i>Transfer learning</i>	37
4.3. Resultados del entrenamiento de modelos.	39
4.4. Resultados de la validación de modelos.	40
4.5. Resultados del testeo de modelos.	42
4.6. Respuesta del modelo RGB en función del desarrollo y con terreno anegado.	42
4.7. Respuesta del modelo FC en función del desarrollo.	43
4.8. Estimación del tiempo para la tarea de conteo.	48
A.1. Tiempo de cómputo para procesamiento en lote.	53
A.2. Detecciones para un mismo sector a distintas alturas.	54
D.1. Especificaciones de la GPU NVIDIA Tesla T4.	65

***Dedicado a mi familia,
por el apoyo incondicional.***

Capítulo 1

Introducción general

En este capítulo se aborda el problema del conteo de plantas en cultivos de arroz, la motivación para eficientizar esta tarea, las soluciones disponibles actualmente en el mercado, y los objetivos y alcance del presente trabajo.

1.1. Introducción al muestreo de cultivos

El conteo de plantas en cultivos es una práctica esencial en la agricultura, ya que proporciona información crítica sobre la densidad de siembra, la salud de los cultivos o el rendimiento esperado. Tradicionalmente, este proceso se ha realizado mediante métodos de muestreo manual, en los que el agricultor toma muestras de cada lote, cuenta las plantas presentes en parcelas representativas, para luego estimar la cantidad total mediante cálculos estadísticos.

Estos métodos presentan gran variabilidad en sus resultados, dependiendo de los criterios utilizados por el especialista al momento de seleccionar las parcelas, por lo que la precisión dependerá del sesgo introducido con esta decisión. Su experiencia y pericia son fundamentales para asegurar una medición representativa.

Por otro lado, el conteo es un proceso exigente que demanda una inversión de tiempo considerable y puede requerir jornadas completas de trabajo, dado que es necesario recorrer toda la extensión del lote, deteniéndose en múltiples ocasiones para tomar muestras. A esto se suma el tiempo necesario para el traslado hasta el campo y el impacto de variables exógenas, como las condiciones meteorológicas y la transitabilidad del terreno, así como de los caminos de acceso, que pueden prolongar considerablemente el tiempo requerido por esta tarea.

1.2. Motivación

Adecoagro sembró en la campaña 2024/25 aproximadamente 65 000 ha de arroz [1], distribuidas en cuatro campos en Argentina (provincias de Santa Fe, Corrientes y Entre Ríos) y dos en Uruguay (en la ciudad de Melo, departamento de Cerro Largo) [2], e incrementa anualmente las hectáreas sembradas un 5 % de media. Para la empresa, el negocio de arroz ha representado durante los últimos 5 años alrededor del 15 % del EBITDA (Beneficios antes de intereses, impuestos, depreciaciones y amortizaciones) y el 20 % de sus ventas, llegando a alcanzar un 35,6 % durante 2023 [3]. Por tanto, para ellos es crucial poder dar un seguimiento óptimo a estos cultivos.

Para lograr atender tal cantidad de hectáreas, Adecoagro cuenta con una gran red de profesionales distribuidos por todo el país, potenciados por el uso de drones, imágenes satelitales, datos meteorológicos y trazabilidad de los cultivos, que pueden ser consultados a través de su plataforma.

Cada encargado de lotes cuenta con dos ayudantes y tiene asignadas de 3 a 4 mil hectáreas, equivalentes a alrededor de 20 lotes (cada uno se extiende por 150 a 300 ha). Realizar un conteo de plantas con métodos tradicionales puede demorar un estimado de 2 horas por hectárea, dependiendo de la extensión del lote, las condiciones climáticas y del terreno, y la cantidad de muestras que se tomen, pudiendo desplazarse dentro del lote caminando o en motocicleta. Es decir, por cada lote se requieren entre 300 a 600 horas-hombre de personal capacitado. Puede verse que este proceso es difícilmente escalable y tiene gran impacto económico para la empresa.

Por el contrario, al realizar un muestreo con drones se puede disminuir considerablemente el tiempo requerido, ya que se elimina la necesidad de ingresar al lote, porque el operario puede cargar el plan de vuelo predefinido y realizarlo de forma autónoma desde su vehículo. De esta forma, se pueden capturar todas las imágenes necesarias para medir un lote entero en menos de 30 min. Además, es sencillo escalar el número de muestras capturadas de ser necesario.

Con esta solución, se logra automatizar el proceso de conteo completo, reducir considerablemente el tiempo, facilitar y agilizar la movilidad dentro del campo, minimizar la capacitación necesaria para desempeñar la tarea y eliminar el sesgo introducido por el operario.

Incluyendo el tiempo de carga y procesamiento de imágenes por el *pipeline* de IA, se podría completar el conteo de plantas en 1 hora utilizando la solución desarrollada. Esto representa un potencial aumento de la productividad de 8 veces.

Al mejorar la eficiencia, reducir el uso de recursos y obtener datos más precisos a menor costo, se podría incrementar el número de conteos anuales por lote, ya que esta solución permite brindar un monitoreo regular a los cultivos y realizar una gestión más eficiente y sostenible.

1.3. Estado del arte

Actualmente, existen diversas soluciones para el conteo automático de plantas mediante técnicas de visión por computadora en distintos tipos de cultivos extensivos, como maíz, trigo o soja. Sin embargo, no se encuentra disponible en el mercado un servicio similar para cultivos de arroz. Según la investigación realizada en el marco del presente estudio, no se han identificado proveedores a nivel internacional que ofrezcan una solución a este problema.

Actualmente, los servicios de análisis para el cultivo de arroz incluyen:

1. Informe de estrés de las plantas: para visualizar las áreas poco saludables del campo y abordar los problemas en una etapa temprana, evita sufrir pérdidas significativas.
2. Informe de la densidad promedio de plantas: para identificar las zonas con una densidad de plantas por debajo de la media y aplicar fertilizantes de forma precisa.

3. Informe de detección de malezas: para detectar zonas afectadas y realizar una aplicación variable de herbicida.
4. Informe de rendimiento: basado en la medición del volumen de la panícula, permite estimar el rendimiento potencial del cultivo.

En el ámbito académico, destacan numerosas investigaciones que han aplicado modelos de visión artificial (*Computer Vision*, CV) en el conteo automático de plantas para diversos cultivos, con el objetivo de comparar el número de plantas germinadas con la cantidad de semillas plantadas y determinar así la efectiva densidad del cultivo.

Tradicionalmente se han aplicado distintas técnicas de visión artificial en agricultura, especialmente la utilización de espacios de color. Dada la naturaleza del problema, en el que el color verde es preponderante en las imágenes de plantas, la utilización de índices de vegetación basados en espacios de color permite discriminar fácilmente plantas de fondo. García-Martínez et al. aplicaron correlación cruzada normalizada (NCC) junto a espacio de color CIELAB para contar plantas de maíz [4]. Valente et al. utilizaron redes convolucionales junto con índices de vegetación (*Excess Green*) y Otsu para identificar y contar plantas de espinaca [5].

Es habitual trabajar con imágenes de alta resolución capturadas por vehículos aéreos no tripulados (*Unmanned Aerial Vehicle*, UAV), popularmente conocidos como drones, ya que permiten abarcar grandes extensiones de cultivo de forma práctica y económica.

A lo largo de los años, se han implementado distintas arquitecturas CNN para detección y conteo de plantas. Madec et al. exploraron la utilización de Faster R-CNN y TasselNet para el conteo de espigas de trigo [6]. Xu et al. aplicaron Mask R-CNN junto a YOLOv5 para contar hojas en plántulas de maíz [7]. También, Ammar et al. compararon el desempeño de Faster R-CNN contra EfficientDet para el conteo de palmas [8]. Wang et al. utilizaron RetinaNet para la detección y conteo de espigas de maíz [9]. Además, Karami et al. propusieron el uso de CenterNet para realizar el conteo automático de maíz con bajo volumen de datos (*few-shot learning*) [10].

Sin embargo, el uso imágenes de drones presenta un desafío adicional, ya que comúnmente los objetos a detectar son pequeños, lo que dificulta la utilización de modelos estándar. Para solucionarlo, Wang et al. incorporó el filtro Kalman al modelo YOLOv3 [11]. Por su parte, Luo et al. combinaron K-means++ y Soft-NMS a YOLOv3, con el mismo fin [12].

Además, en muchos casos se utilizan redes de convolución combinadas con etapas adicionales que complementan su funcionamiento al facilitar la extracción de características (*features*) y permitir el aprendizaje del modelo. Tatini et al. propusieron agregar una capa adicional a YOLOv4 de tipo SUFF lo que permite extraer mejores mapas de características para campos de arroz [13]. Por otra parte, Yang et al. combinaron GhostNet con YOLOv5 y aplicaron *Efficient Intersection over Union* (EIoU) para optimizar el rendimiento [14]. En contraste, Luo et al. reemplazaron el *backbone* de YOLOv5 con ASRes2Net e incluyeron *Efficient Channel Attention* (IECA) para mejorar la precisión en la clasificación [15].

Estudios recientes han intentado abordar el problema de conteo de plantas para el caso específico de los cultivos de arroz. En este sentido, Yeh et al. propusieron la utilización del modelo YOLOv4 reemplazando la función de activación ReLU por Mish con excelentes resultados [16]. Por otro lado, Yao et al. proponen la utilización de P2PNet-EFF con un *backbone* de atención multiescala eficiente (EMA) para lograr un conteo preciso de plantas [17].

Si bien estas investigaciones han contribuido a expandir el campo del conocimiento, con buenas métricas de precisión y *recall*, es fundamental profundizar en este dominio para poder ofrecer soluciones prácticas y fiables. Es crucial que las soluciones provistas se adapten a las particularidades de los cultivos locales y las características desafiantes que presenta la agricultura intensiva, que tiene una densidad de plantas considerablemente mayor a la de los estudios académicos realizados generalmente en pequeños lotes de ensayo.

1.4. Objetivos y alcance

El objetivo del presente trabajo es automatizar el conteo de plantas en cultivos de arroz para reducir la utilización de recursos humanos, en comparación con los requeridos por métodos tradicionales de muestreo, que son intensivos en tiempo y requieren la presencia de personal especializado en el campo.

Al automatizar este proceso, se espera eficientizar la atención de esta necesidad e incrementar el número de conteos por cosecha, lo que permitirá dar seguimiento regular a los cultivos de forma eficiente y a menor costo.

El trabajo incluye:

- Captura en campo de imágenes aéreas de alta resolución.
- Procesamiento y clasificación de imágenes.
- Etiquetado de imágenes para la creación del dataset.
- Evaluación de diferentes arquitecturas de visión artificial.
- Desarrollo del *pipeline* de IA:
 - Detección de objetos-planta en imágenes.
 - Trazado de línea de distribución del cultivo.
 - Discriminación de predicciones entre plantas y maleza.
 - Conteo del total de plantas por lote.
 - Estimación del área cubierta por la fotografía.
 - Cálculo de la densidad por hectárea.
- Optimización del resultado a través del análisis de métricas.

Capítulo 2

Introducción específica

En este capítulo se presentan las técnicas y herramientas utilizadas para desarrollar el presente trabajo.

2.1. Vehículos aéreos no tripulados (UAV)

La tecnología de vehículo aéreo no tripulado (*Unmanned Aerial Vehicle*, UAV) representa una revolución en múltiples industrias, incluyendo un papel cada vez más crucial en la agricultura. Estos dispositivos están equipados con una gran variedad de sensores, cámaras y sistemas de navegación (como el GPS), que les permiten volar de forma autónoma (siguiendo un plan de vuelo predefinido) o ser dirigidos de forma remota mediante radiocontrol.

En el ámbito agrícola, los drones ofrecen una perspectiva aérea invaluable para la monitorización de cultivos, porque permiten la detección temprana de enfermedades, plagas o estrés hídrico a través de fotografías y videos de alta resolución, imágenes multiespectrales o térmicas. Pueden capturar gran cantidad de datos que facilitan la toma de decisiones precisas sobre la aplicación de fertilizantes, pesticidas o riego. Esto conduce a la optimización de los recursos y a la mejora del rendimiento. Además, existen drones de gran envergadura equipados para la siembra de precisión y la fumigación selectiva.

2.1.1. Características de los drones utilizados

Para la captura de imágenes se utilizaron drones DJI Mavic 2 Pro [18]. Este modelo posee una autonomía estimada de 30 min en condiciones ideales (sin viento, a una velocidad constante de 25 km/h) y alcanza una velocidad máxima de 72 km/h (~ 20 m/s). Estas especificaciones le permiten recorrer completo un lote de 200 ha y capturar las fotografías muestrales sin requerir una recarga o cambio de batería.

El dron cuenta con sistema de geolocalización dual GPS + GLONASS que mejora la estabilidad del vuelo y la precisión de la ubicación. La precisión de altitud es de $\pm 0,5$ m volando solo con GPS y $\pm 0,1$ m al utilizarse también los sensores visuales.

El DJI Mavic 2 Pro está equipado con una cámara Hasselblad L1D-20c con sensor CMOS de 1 in con una resolución de 20 MP. El lente tiene una distancia focal equivalente a 28 mm, una apertura ajustable de $f/2.8$ a $f/11$, y enfoca desde 1 m hasta infinito. Las fotografías pueden guardarse en formato JPEG o DNG (RAW), con una resolución máxima de 5472×3648 px. El sistema de estabilización de la

cámara está basado en un *gimbal* mecánico de 3 ejes con precisión de $\pm 0,01^\circ$, que permite obtener imágenes nítidas y sin distorsión durante el vuelo.

2.2. Tratamiento de datos

Para el presente trabajo se elaboró un set de datos original a partir de imágenes provistas por el cliente. Además, se aplicaron técnicas de aumento de datos para enriquecer dataset y mejorar la generalización del modelo.

2.2.1. Etiquetado de imágenes

Para el etiquetado de imágenes se utilizó la herramienta Roboflow, con el estándar de etiquetas YOLOv8 PyTorch TXT, una modificación del formato YOLO Darknet [19]. Según este formato, el dataset se organiza en una estructura de carpetas (`train`, `valid`, `test`) y subcarpetas (`images`, `labels`), indicadas dentro del archivo `data.yaml`, donde también se detallan el número de clases y sus nombres. Para cada imagen del dataset existe un archivo `.TXT` del mismo nombre, ubicado en la carpeta correspondiente (ej: `../train/labels`).

Cada línea del archivo representa una etiqueta de la forma:

```
class_id center_x center_y width height
```

La etiqueta detalla la clase y las coordenadas normalizadas (de 0 a 1) con la posición relativa del centroide (`center_x`, `center_y`), el ancho (`width`) y el alto (`height`) del *bounding box*.

2.2.2. Técnica de aumento de datos (*Data Augmentation*)

Data augmentation es un conjunto de técnicas que permiten generar nuevas imágenes para enriquecer el dataset, lo que mejora la generalización. A diferencia de los “datos sintéticos”, que son completamente nuevos, los “datos aumentados” se crean al aplicar transformaciones al set original [20].

Particularmente en el dominio de la visión computacional, estas técnicas pueden simular diversos escenarios que el modelo podría encontrar en el mundo real y ayudarlo a desempeñarse mejor en diferentes condiciones, independientemente de la orientación, tamaño o condiciones de iluminación de la fotografía [21]. Pueden verse ejemplos de las transformaciones más comunes en la figura 2.1.



FIGURA 2.1. Ejemplo de aumentación de datos ¹.

¹ Adaptación de la imagen del *paper* <https://www.mdpi.com/2077-0472/14/3/331>

2.3. Modelos de inteligencia artificial utilizados

La utilización de modelos de aprendizaje profundo (*Deep Learning*, DL) ha demostrado ser de gran utilidad en la agricultura por responder mejor a la variabilidad de los datos de entrada (condiciones lumínicas y climáticas, características del terreno y el cultivo, malezas locales, estado de crecimiento de la planta, etc.). Esto permite además la aplicación de *data augmentation* y *transfer learning* [10].

2.3.1. Arquitecturas convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son la base de modelos de visión artificial para clasificación, detección y segmentación de objetos. Operan en dos etapas: extracción de características y clasificación.

Su fortaleza radica en la extracción de características (*feature extraction*) de forma autónoma mediante capas convolucionales, que aplican filtros (*kernels*) píxel a píxel para identificar patrones y crear mapas de características [22]. Las subsiguientes capas de *down-sampling* reducen la dimensionalidad (*pooling*) y abstraen las principales características. La concatenación de estas estructuras permite identificar patrones complejos.

Finalmente, como puede verse en la figura 2.2, una capa de aplanamiento (*flatten layer*) transforma el tensor multidimensional en un vector unidimensional, que alimenta las capas *fully-connected* de la etapa de clasificación, que funcionan como una red neuronal clásica.

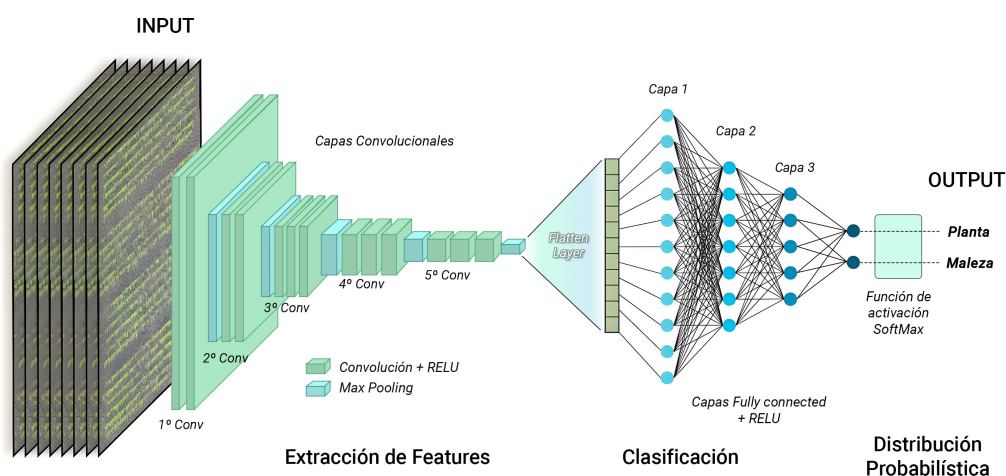


FIGURA 2.2. Estructura característica de una red CNN.

2.3.2. Arquitectura YOLO

El modelo de visión computacional YOLO, presentado por Redmon et al. [23] en el artículo «*You Only Look Once: Unified, Real-Time Object Detection*» en 2015, revolucionó la visión artificial al permitir la detección de objetos en tiempo real. A diferencia de arquitecturas previas (R-CNN, Fast R-CNN, etc.) que usaban un proceso de dos etapas (propuestas de región y clasificación), YOLO unificó todo en un solo paso, reduciendo drásticamente el tiempo de inferencia.

YOLO también ofrece invarianza a la traslación y al escalado desde la v3 con *Feature Pyramid* [24] y a la rotación al aplicar *data augmentation* (especialmente en la v8) [25]. Las sucesivas versiones han mejorado la precisión, el rendimiento y la velocidad de entrenamiento e inferencia [26].

En el contexto de la detección de plantas en entornos productivos, la característica de YOLOv8 de ser “*anchor-free*” es crucial ya que simplifica el entrenamiento y lo vuelve robusto ante variaciones de forma y escala de los objetos [27]. Esto le permite reconocer plantas visualmente diferentes a las del conjunto de entrenamiento (debido al estado fenológico, densidad de plantas o posición de las hojas).

2.3.3. Técnica de *Fine-Tuning* (Transfer Learning)

La transferencia de aprendizaje (*Transfer Learning*) es un conjunto de técnicas que permite trasladar el aprendizaje de un modelo a otros dominios específicos.

La técnica de ajuste fino (*Fine-Tuning*), en particular, permite cargarle a cada neurona de la red los pesos de modelos preentrenados y utilizarlos como punto de partida para un nuevo entrenamiento. Esto facilita que unas capas aprendan las características del dataset, mientras otras se mantienen inmutables (proceso conocido como *freeze* o “congelación de parámetros”).

Las ventajas principales de esta técnica son: optimización, ya que reduce el esfuerzo computacional al usar modelos pre-entrenados; transferencia cognitiva, permitiendo que modelos entrenados para una tarea funcionen bien en otros dominios; adaptabilidad, facilitando la personalización para contextos específicos; y reducción de requisitos de datos, logrando alta precisión con conjuntos de datos más pequeños [28].

2.4. Algoritmo DBSCAN

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) es un robusto y eficiente algoritmo de *clustering* con aprendizaje no supervisado. Como se ve en la figura 2.3, supera ampliamente a alternativas como *K-means* al no requerir conocimiento previo del número de *clusters* [29]. Su capacidad para identificar agrupaciones de datos y discriminar el ruido (puntos aislados) lo hace ideal para datos con alta variabilidad, como fotografías aéreas de cultivos.

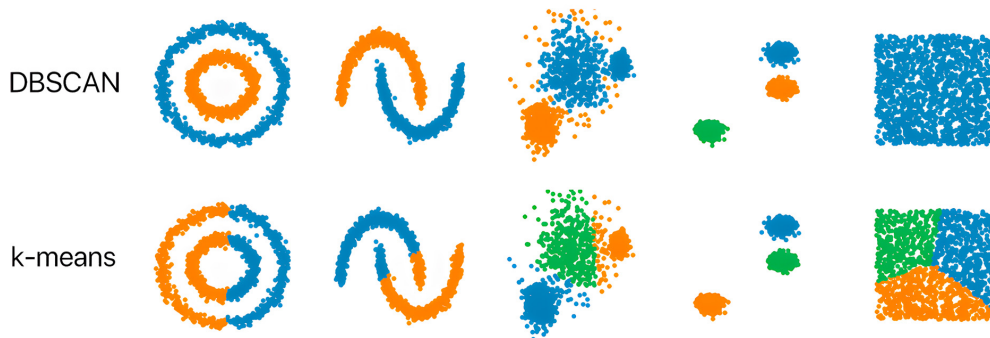


FIGURA 2.3. Comparación entre algoritmos DBSCAN y *K-means* ².

²Imagen tomada del repositorio <https://github.com/NSHipster/DBSCAN/>

Para su implementación, DBSCAN requiere dos parámetros clave: épsilon (ϵ), distancia máxima para que dos puntos sean considerados vecinos, y puntos mínimos (minPoint), umbral de puntos necesarios para formar un *cluster* [30].

El algoritmo selecciona un punto arbitrariamente y verifica cuántos vecinos existen dentro del radio ϵ . Si excede minPoint , esos puntos se etiquetan como parte de un mismo *cluster*. Puede verse representado en la figura 2.4. Este proceso se expande iterativamente, se trazan nuevos círculos desde los puntos detectados para encontrar nuevos vecinos dentro de ϵ y recorrer progresivamente todo el *cluster* [31]. Aquellos puntos que no cumplen las condiciones de distancia y cantidad son clasificados como “ruido” (se les asigna el valor -1).

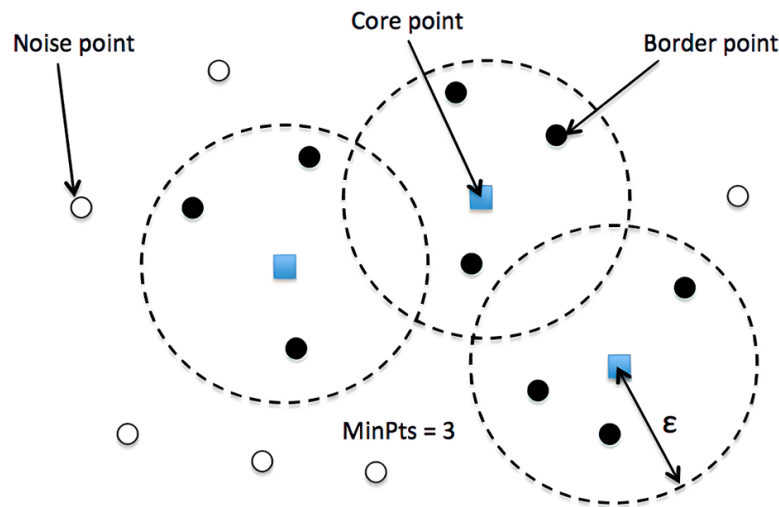


FIGURA 2.4. Ejemplo del proceso de búsqueda de puntos vecinos y detección de outliers³.

La búsqueda de vecinos cercanos puede hacerse con fuerza bruta (para datos dispersos), *ball tree* o árbol k-d (con datos densos) [30].

2.5. Índices de vegetación: *Excess Green*

Los índices de vegetación son herramientas cruciales en la agricultura moderna para monitorear la salud y cobertura de los cultivos de forma no destructiva. Se basan en combinaciones matemáticas de bandas espectrales capturadas por sensores remotos, como satélites o drones. Al analizar la reflectancia en espacios de color como RGB, HSV o Lab, estos índices resaltan las propiedades biofísicas de las plantas, permitiendo detectar estrés, estimar rendimientos y diferenciar cultivos de malezas, optimizando así la gestión agrícola [32].

Un índice particularmente útil es el exceso de Verde (*Excess Green*, ExG), un algoritmo de espacio de color aplicado a imágenes RGB que enfatiza los píxeles con alta información en el canal verde [33]. Esto es invaluable en agricultura para medir la cobertura del cultivo y evaluar la salud de las plantas, diferenciando material orgánico (plantas, malezas) del resto de los elementos en la imagen [34].

³Imagen tomada de el artículo de KDnuggets <https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html>

Se calcula como $ExG = 2G - R - B$, donde G, R y B representan los valores de las bandas Verde, Roja y Azul, respectivamente. Es fundamental aplicar escalado Min-Max [35] a cada canal RGB antes del cálculo para homogeneizar sus rangos y reducir la influencia de la iluminación variable, asegurando así un índice más robusto y consistente entre distintas imágenes.

El resultado es una imagen en escala de grises donde los valores más altos corresponden a la vegetación. Comúnmente, se aplica un umbral para segmentar la vegetación, creando una máscara binaria (blanco para plantas, negro para el fondo) que funciona como un filtro para recortar las plantas de la imagen original. La elección del umbral varía según las condiciones de iluminación y las características de las imágenes.

2.6. Análisis de componentes principales (PCA)

El análisis de componentes principales (PCA) es una técnica estadística utilizada para la reducción de dimensionalidad, especialmente en conjuntos de datos con muchas variables interrelacionadas, como imágenes multiespectrales [36]. Se busca simplificar la información al transformar las bandas correlacionadas en un nuevo conjunto de componentes principales no correlacionados.

El primer componente principal captura la mayor varianza (información), mientras que los subsiguientes capturan la varianza restante, siendo todos ortogonales entre sí [37]. Esta técnica permite la extracción de características (*feature extraction*) sin pérdida significativa de información, lo que hace que los componentes principales sean ideales como nuevas características para algoritmos de clasificación o segmentación [38]. Además, PCA contribuye a la reducción de ruido en las imágenes, ya que los componentes con menor varianza suelen contener el ruido, lo que permite obtener una representación más limpia al descartarlos [39].

2.7. Ecualización adaptativa del histograma (CLAHE)

La ecualización adaptativa del histograma con límite de contraste (CLAHE) es una técnica avanzada de mejora del contraste en imágenes digitales [40]. A diferencia de los métodos globales, CLAHE opera localmente, dividiendo la imagen en pequeñas regiones (ej., 8x8 píxeles) para aplicar transformaciones de contraste de manera independiente. Esto es especialmente beneficioso en imágenes con iluminación o contraste variables.

La técnica busca redistribuir los niveles de intensidad de los píxeles para lograr un histograma más uniforme, expandiendo el rango dinámico y revelando detalles ocultos [41]. Después de dividir la imagen en bloques, CLAHE calcula el histograma de cada uno, aplicando un límite de contraste para evitar la amplificación excesiva de ruido. Finalmente, se usa interpolación bilineal entre los bloques para asegurar transiciones suaves y evitar discontinuidades visuales en la imagen mejorada.

Capítulo 3

Diseño e implementación

En este capítulo se profundiza en la solución desarrollada para el problema. Se detallan las características de los cultivos, la captura de imágenes y la arquitectura general del sistema, así como el funcionamiento de cada una de las etapas.

3.1. Arquitectura del sistema

La solución propuesta se implementó con un *pipeline* de inferencia que puede verse en detalle en la figura 3.1. Este sistema combina una etapa de preprocesamiento inicial, la instancia del modelo de CV y el posprocesamiento de las detecciones para identificar las líneas y calcular la densidad.

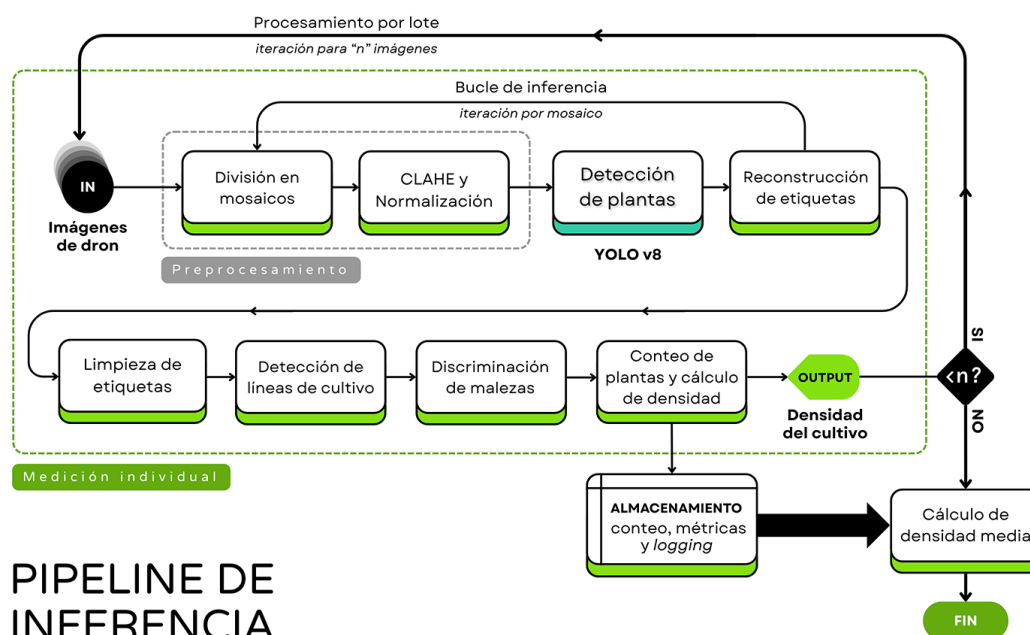


FIGURA 3.1. Diagrama del *pipeline* de inferencia.

3.2. Captura de imágenes

El dataset fue construido a partir de las imágenes provistas por el cliente. Por tratarse de una solución específica para un entorno productivo determinado, esta responde a las características particulares de los cultivos tratados, la localización geográfica y las necesidades propias del cliente para el que fue diseñado.

3.2.1. Características del cultivo

Las imágenes fueron capturadas en dos establecimientos de la empresa Adecoagro, ubicados en la provincia de Corrientes, Argentina (tabla 3.1); sembrados entre el 19 de agosto y el 5 de noviembre del 2024 con distintas variedades de arroz: IRGA 424, IRGA 424 CL, IC 126, IC 109, IC120, IC131 y Kira.

TABLA 3.1. Información sobre los establecimientos.

Establecimiento	Ubicación	Coordenadas	Lotes
Doña Marina	Yahapé, Corrientes	27° 26' S / 57° 40' W	209, 503, 212
La verde	San Isidro, Corrientes	29° 43' S / 57° 28' O	211, 212

3.2.2. Plan de vuelo de drones

En una primera instancia, se realizaron vuelos exploratorios que se muestran en la tabla 3.2, para capturar imágenes de distintas etapas de desarrollo de la planta (temprano y medio) a distintas alturas (con un rango de 1 a 10 m de altura, a intervalos de 0,5 m) y ángulos de 30°, 45°, 50°, 60° y 90° (medido respecto al plano horizontal del suelo).

En total se realizaron 4 paradas en distintos sectores del lote 209 (desarrollo temprano) y 503 (desarrollo avanzado), y una adicional para lote 212 (anegado con desarrollo temprano). Para cada parada se seleccionaron de 2 a 4 puntos representativos como muestras y se realizaron las fotografías para las alturas y ángulos mencionados. Además se incorporaron 1 o 2 aros de muestreo por imagen, con los que se realizó un conteo manual tradicional para utilizar como *ground truth* (verdad fundamental).

TABLA 3.2. Primera captura de imágenes (2/10/2024).

Cantidad	Lote	Altura (m)	Ángulo (°)	Característica
140	209	1 a 10	90	Desarrollo temprano
112	503	1 a 10	90	Desarrollo avanzado
13	209 503	1 a 5	60	
13	209 503	1 a 5	30	
12	209 503	1 a 5	45	
3	212	5 y 10	90	Anegado
2	209	1,5	50	

Luego, se verificó la calidad de las imágenes capturadas, ordenándolas en carpetas por altura y ángulo. Se descartaron un total de 52 imágenes defectuosas en las que se detectaron los siguientes problemas:

- dificultad para enfocar en las imágenes de baja altitud (1 m),
- poca definición en algunas imágenes a mayor altitud (>5 m),
- imposibilidad de capturar imágenes útiles en situaciones de fuertes vientos.

Se realizó un conteo *a posteriori* para los aros que fue comparado con el conteo realizado *a priori* para calcular así el Error Cuadrático Medio (RMSE) y el R^2 a partir de las ecuaciones 3.1. Como puede observarse en la figura 3.2, a partir de los 6 m

la resolución de la imagen (píxel/cm) se vuelve un factor crítico que imposibilita la correcta identificación de plantas.

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}} \quad R^2 = 1 - \frac{SC_{res}}{SC_{tot}} \quad (3.1)$$

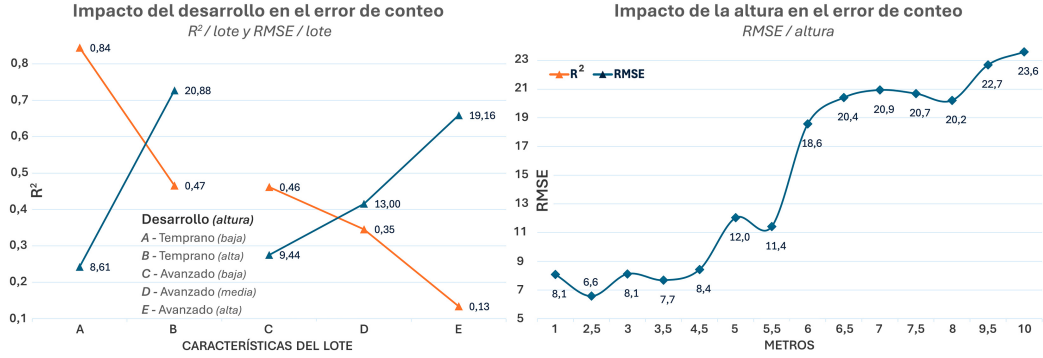


FIGURA 3.2. Error en el conteo de plantas en función de la altura.

La figura 3.3 evidencia la importancia de realizar el conteo en una etapa temprana del desarrollo de la planta para poder garantizar un buen resultado. Como puede apreciarse, solo para el lote 209 se obtuvo un conteo adecuado en los 2 y 4 metros. Para la planta en un estadio avanzado de desarrollo se vuelve más difícil reconocer plantas superpuestas. Es por esto que se decidió trabajar con imágenes a 3,5 m ya que ofrecen el mejor balance entre resolución en los detalles y área del cultivo abarcada.

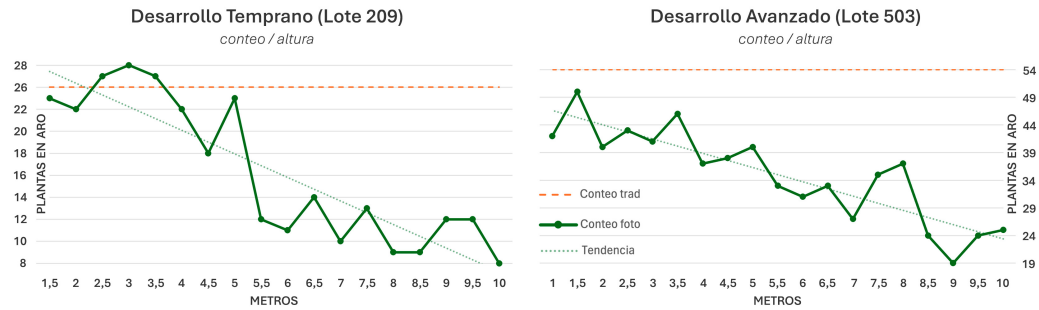


FIGURA 3.3. Error de conteo por lote en función de la altura.

Luego, con base en esos resultados se decidió realizar vuelos a menor altitud (entre 3 y 4 metros) para lograr un buen balance entre número de plantas y resolución por píxel. En el Apéndice C se incluye el protocolo de vuelo enviado al cliente para capturar las imágenes del dataset.

En una segunda etapa, el cliente proveyó 63 imágenes adicionales de plantas en etapa de desarrollo tardío (lotes 211 y 212), con alturas de 5, 10, 15 y 30 m y ángulos de 90° y 60°, como se muestra en la tabla 3.3. 22 de las muestras incluyen el aro de medición y 4 imágenes poseen la característica de incluir huellas del personal (mismo sector a distintas alturas). Por la naturaleza del problema, solo fueron realizadas pruebas de desarrollo con las de menor altitud (5 m) anguladas y cenitales, para realizar algunas pruebas durante el desarrollo.

TABLA 3.3. Segunda captura de imágenes (12/12/2024).

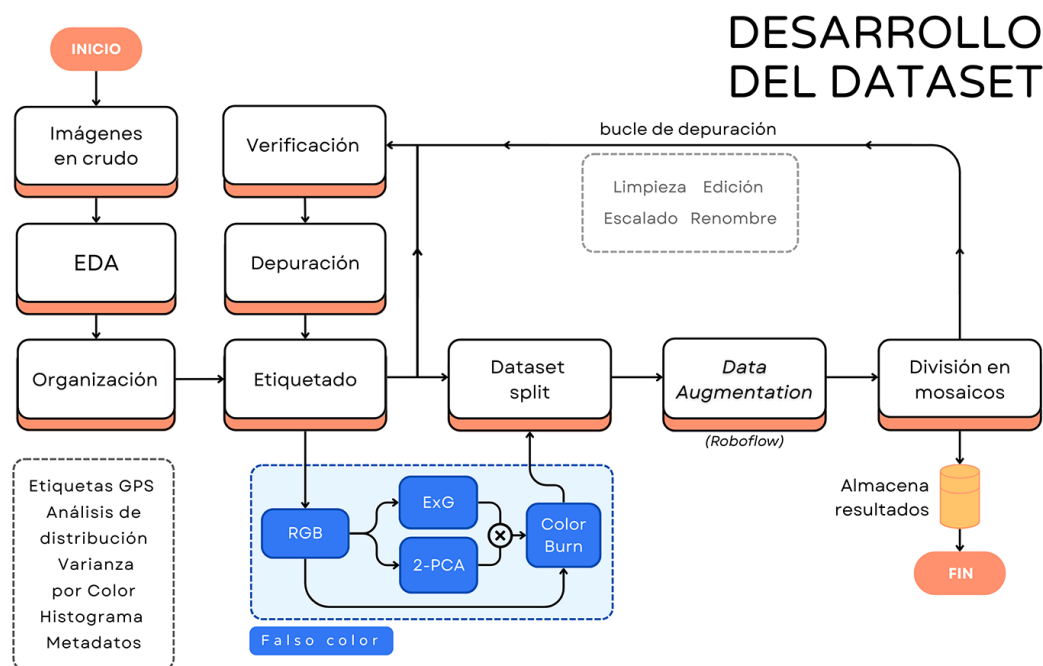
Cantidad	Lotes	Altura (m)	Ángulo
11	211 212	5	90°
12	211 212	10	90°
11	211 212	15	90°
10	211 212	30	90°
18	211 212	5 a 30	60°

Finalmente, estas imágenes no fueron incorporadas al dataset de entrenamiento ya que, por causa del estadio de desarrollo, las plantas se encontraban superpuestas y no era posible realizar un etiquetado fiable.

3.3. Construcción del dataset

Dentro del marco de este trabajo, se etiquetaron 24 imágenes en distintas etapas de desarrollo de plantas, con altura de vuelo y ángulos de la cámara particulares. Durante el desarrollo del dataset se asignaron un total de 49 285 *bounding boxes* que demarcan plantas de arroz, *clusters* de plantas y objetos extraños.

Se desarrolló un conjunto de *scripts* que facilitan el análisis y procesamiento de datos, etiquetas, preprocesamiento de imágenes, *data augmentation* y depuración. Estos pueden ser vistos en detalle en la figura 3.4.

FIGURA 3.4. Diagrama de *scripts* para el desarrollo del dataset.

El dataset final está compuesto por 14 imágenes con vista cenital de cultivos de arroz en etapa de desarrollo temprano (Lote 209) y avanzado (Lote 503), tomadas a una altura de 3,5 metros con un ángulo de 90°. Son imágenes a color de gran resolución, con dimensiones de 5472×3078 píxeles, en formato JPEG con 3 canales de color RGB, sin canal alfa. Cada archivo pesa aproximadamente 5 MB.

Las soluciones propuestas por este trabajo se nutren de dos datasets alternativos:

- Imágenes a color: conformado por las imágenes originales RGB.
- Imágenes de falso color: se implementan índices de vegetación (*Excess Green*).

3.3.1. Clases y estrategia de etiquetado

Si bien se trata de un problema de clasificación binaria (planta/fondo), por tratarse de imágenes con una gran cantidad de etiquetas (~2000 por imagen), se decidió adoptar una estrategia de etiquetado con cuatro clases: `plant-weed`, `plant-occluded`, `plant-cluster` e `intruder`. Esto se hizo con el objetivo de reconocer visualmente áreas problemáticas de las imágenes y permitir la implementación a futuro de otras estrategias complementarias que ayuden en la resolución del problema de conteo.

Función de cada clase:

- `plant-weed`: son las etiquetas ideales, identifican todas aquellas plantas que se ven claramente y pueden ser etiquetadas con un alto nivel de confianza.
- `plant-occluded`: indica aquellas etiquetas en las que se tiene un bajo nivel de confianza de etiquetado (plantas ocultas, superpuestas, tapadas por rastrojo, etc.). El principal objetivo es poder identificar rápidamente zonas problemáticas.
- `plant-cluster`: señala las zonas de alta aglomeración de plantas (denominados *clusters*) en las que se vuelve prácticamente imposible el correcto etiquetado dado el elevado nivel de superposición. Se agregó con la idea de poder aplicar alguna heurística durante el conteo o entrenar modelos en paralelo que detectaran los *clusters* y aplicaran alguna estrategia (estas soluciones no han sido implementadas para el presente trabajo).
- `intruder`: identifica los pocos objetos extraños hallados en las imágenes (generalmente algún tipo de maleza o material inorgánico). Se utilizó para poder hacer un seguimiento de estos y poder localizarlos de forma rápida.

Para el entrenamiento de los modelos de este trabajo, las clases “`plant-weed`” y “`plant-occluded`” fueron unificadas bajo una única clase, mientras que las clases “`plant-cluster`” e “`intruder`” fueron omitidas. Es decir, el modelo YOLO fue entrenado para realizar detecciones uniclase (plantas).

Gestión de nombres de archivos

A las imágenes etiquetadas con Roboflow se les agrega de forma automática un ID interno de 32 caracteres (formado por caracteres alfanuméricos) que se agrega al final del nombre del archivo original, luego de la etiqueta “`rf`”.

Junto al archivo de imagen se almacena un archivo de texto con las etiquetas. Las imágenes se almacenan dentro de la carpeta `images` y las etiquetas en la carpeta `labels`.

A modo de ejemplo:

- Entrada (imagen sin etiquetar): `209_101_24.jpg`

- Salida (imagen etiquetada):

- `images/209_101_24_JPG.rf.8d87d8429...cc299d413.jpg`
- `images/209_101_24_JPG.rf.8d87d8429...cc299d413.txt`

3.3.2. Aumentado de datos

Para el conjunto de entrenamiento, se aplicó *Data Augmentation* de $3\times$ con la herramienta de Roboflow, para generar dos nuevas imágenes por cada una existente. En la tabla 3.4 puede verse el detalle de las transformaciones aplicadas. El criterio utilizado para definir los parámetros fue obtener transformaciones realistas, similares a las imágenes que podría encontrarse el modelo en producción.

Estrategia de *Data Augmentation* de 1 etapa

El proceso consistió en transformaciones espaciales (rotación, espejado y recorte) y de color (saturación, brillo y exposición). Adicionalmente, se incorporó un leve desenfoque para ayudar en la detección en imágenes fuera de foco, problema identificado durante la construcción del dataset. También se agregó una pequeña componente de ruido aleatorio que ayuda en la generalización del modelo, volviéndolo más resiliente ante el ruido del sensor o pequeñas imperfecciones.

TABLA 3.4. Transformaciones de Roboflow.

Transformación	Parámetros
Brillo	$\pm 25\%$
Saturación	$\pm 30\%$
Exposición	$\pm 5\%$
Rotación	$\pm 90^\circ/180^\circ$
Espejado (<i>Flip</i>)	H/V
Recorte (<i>Zoom</i>)	hasta 30%
Desenfoque (<i>Blur</i>)	hasta 2 px
Ruido (<i>Noise</i>)	hasta $0,1\%$

Aplicación para el dataset con falso color

De forma alternativa, para el dataset con falso color, se removieron algunas transformaciones (saturación, brillo y desenfoque) para no limitar el resultado logrado durante el procesamiento de imágenes. Las transformaciones aplicadas fueron:

- Exposición: $\pm 5\%$
- Espejado (*Flip*): H/V
- Rotación: $\pm 90^\circ/180^\circ$
- Recorte (*Zoom*): hasta 30%
- Ruido: hasta $0,1\%$

Estrategia de *Data Augmentation* de 2 etapas

Posteriormente, se decidió aplicar una estrategia más elaborada en 2 etapas, para alcanzar un incremento de $9\times$ en el número de imágenes (también con Roboflow).

1º etapa:

- Brillo: $\pm 25\%$
- Saturación: ± 20
- Espejado (*Flip*): H/V
- Rotación: $\pm 90^\circ/180^\circ$
- Recorte (*Zoom*): hasta 30%

2º etapa:

- Brillo: -5% (reducción)
- Saturación: $\pm 10\%$
- Exposición: $\pm 10\%$
- Rotación gradual: $\pm 15^\circ$
- Deformación (*Shear*): $\pm 10^\circ$ H/V
- Recorte (*Zoom*): hasta 10%
- Desenfoque (*Blur*): hasta $1,5$ px
- Ruido (*Noise*): hasta $0,1\%$

Estrategia de *Data Augmentation* suave

Adicionalmente, la implementación de YOLOv8 de Ultralytics incorpora por defecto un leve aumento de datos con la biblioteca Albumentations [42] que busca simular artefactos visuales del mundo real y mejorar la robustez del modelo al aplicar con una baja probabilidad ($p = 1\%$): desenfoque gaussiano, desenfoque mediano, conversión a escala de grises y CLAHE. Puede verse cada parámetro en detalle en la tabla 3.5. Esta opción puede desactivarse si se desinstala la biblioteca Albumentations antes de iniciar el entrenamiento.

TABLA 3.5. Transformaciones de Albumentations (YOLO *default*).

Transformación	Parámetros
Desenfoque gaussiano (<i>GaussianBlur</i>)	3 a 7 px
Desenfoque mediano (<i>MedianBlur</i>)	3 a 7 px
Escala de grises (<i>ToGray</i>)	$0,299R + 0,587G + 0,114B$
CLAHE	<code>clip_limit: 1 a 4</code> <code>grid_size: (8, 8)</code>

3.3.3. Aplicación de índices de vegetación (dataset de falso color)

Durante el armado del dataset se exploraron diversas aplicaciones de espacios de color, particularmente el uso del índice exceso de verde (*Excess Green*).

***Excess Green* (ExG)**

Se utilizó el índice de vegetación *Excess Green* para generar imágenes en escala de grises que destacan la localización de cada planta dentro de la imagen. La imagen resultante puede verse en 3.5. Para su implementación, se aplicó primero un escalado Min-Max [35] de 0 a 255 para cada canal de color, y luego se procesó el algoritmo estándar.

Reducción dimensional (2-PCA)

Los prometedores resultados al diferenciar visualmente las plantas del fondo mediante imágenes ExG motivaron su integración en la arquitectura YOLOv8. Sin

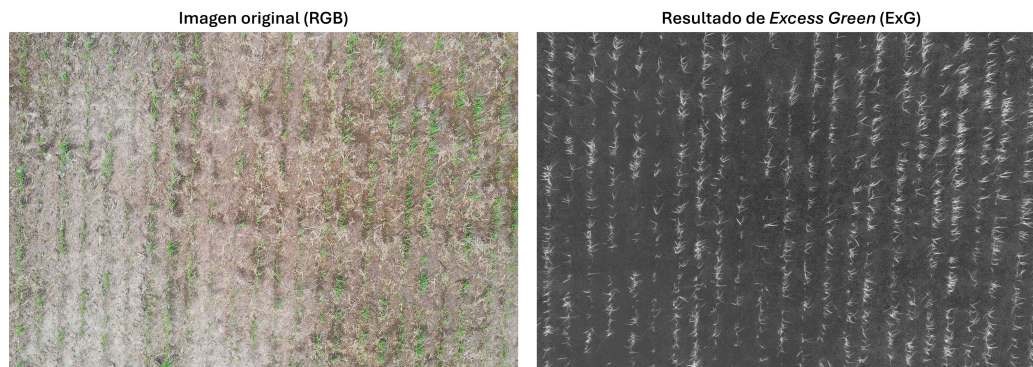


FIGURA 3.5. Resultado de la transformación *Excess Green*.

embargo, dado que esta tradicionalmente procesa imágenes RGB de 3 canales, la imagen ExG (en escala de grises) no podía simplemente añadirse como un cuarto canal alfa (transparencia).

Ante la limitación de tener solo dos canales disponibles (uno para ExG), se realizó un análisis de la distribución de la información por canal en el dataset mediante el cálculo de un histograma promedio (figura 3.6).

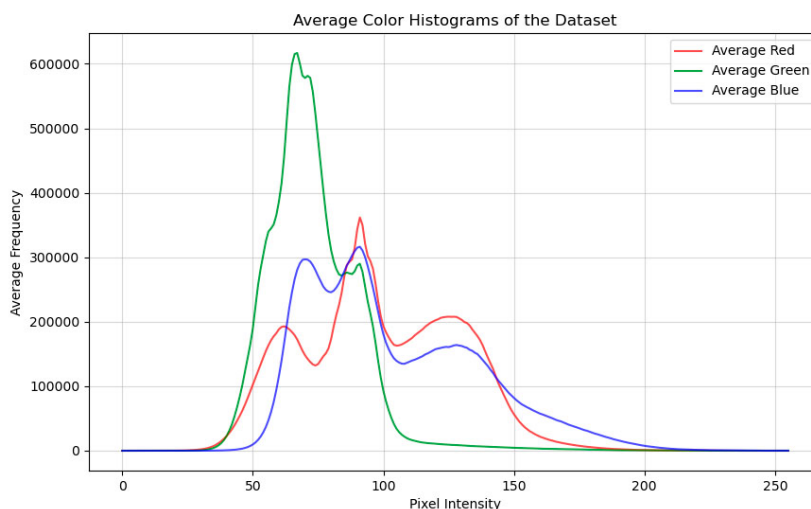


FIGURA 3.6. Histograma promedio por canal para el dataset.

Al observar una contribución significativa de los tres canales RGB originales, se descartó la eliminación completa de uno de ellos. En su lugar, se optó por aplicar una técnica de reducción dimensional, específicamente análisis de componentes principales (PCA), para reducir la información de los 3 canales RGB a 2 dimensiones.

Para implementar PCA se utilizó la biblioteca Scikit-learn (`sklearn`) [43] y se entrenó un modelo con cada imagen, para proyectar las direcciones con mayor varianza en el nuevo espacio de menor dimensionalidad. Para aplicar PCA a las imágenes es necesario, en primer lugar, transformar el vector de imagen del formato estándar (alto, ancho, canales) al de entrada de PCA (alto x ancho, color); esto se llevó a cabo con la biblioteca Numpy [44]. Finalmente, se precisa aplicar escalado Min-Max [35] a la salida de PCA, para que los valores ocupen nuevamente el rango $[0, 255]$.

Como puede verse en la figura 3.7, el segundo componente de PCA guarda mucha similitud con el resultado de ExG (con menor contraste), mientras que el primer componente es prácticamente una representación del fondo sin plantas.

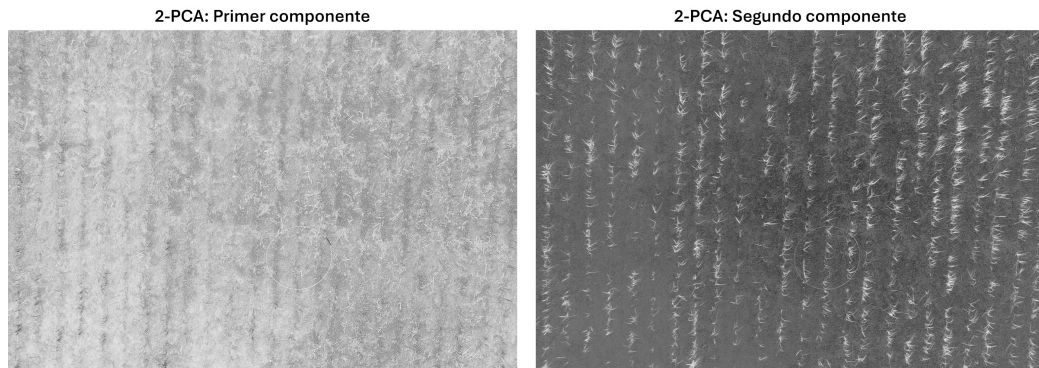


FIGURA 3.7. Resultado de la reducción dimensional con PCA.

Falso color (FC)

Se generó una nueva imagen RGB con falso color al combinar ExG con la reducción dimensional de 2-PCA. Se asigna cada imagen gris a un canal de RGB.

Se estudiaron las distintas combinaciones posibles para asignar cada entrada a cada canal (R, G y B) y se seleccionó la que conservó el color verde para las plantas. Esto se hizo con fines prácticos, por ser más reconocible para la visión humana, ya que para el modelo (no pre-entrenado) no debería representar ninguna diferencia.

La imagen resultante queda compuesta por:

- Canal R: *Excess Green* (ExG)
- Canal G: 2° componente (PC2)
- Canal B: 1° componente (PC1)

Además, de este modo se logra conservar la propiedad de las plantas de ocupar mayormente el espectro verde y rojo.

Como se puede apreciar en la figura 3.8, con esta transformación es mucho más sencillo identificar las plantas para el ojo humano. Sería razonable esperar también una mejora similar para el modelo.

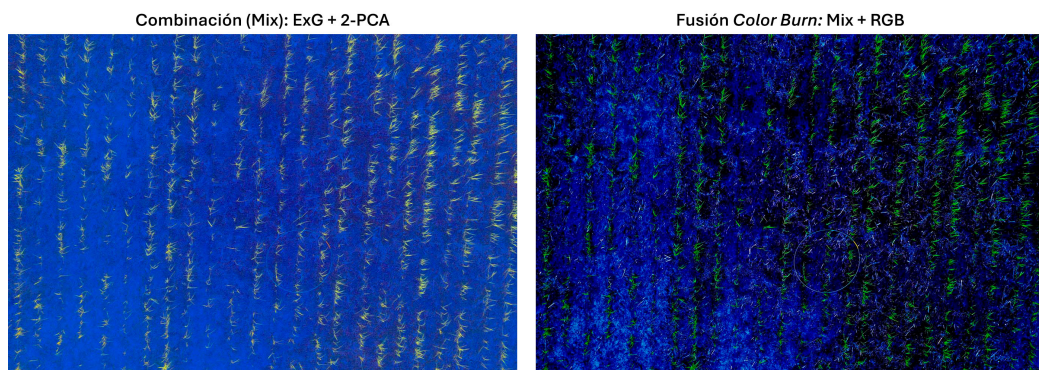


FIGURA 3.8. Resultado de la fusión *Color Burn*.

Subexposición de color (*color burn*)

Se aplicó una última etapa de subexposición de color para fusionar la imagen de falso color con la original (RGB), con el objetivo de conservar la alta resolución de la fotografía del dron y al mismo tiempo incorporar la nueva información aportada por la imagen procesada (ExG+2-PCA), para facilitar así la tarea de detección al modelo.

Color burn oscurece los colores de la capa base (imagen original) basándose en los de la capa de fusión (imagen de falso color). Esta fusión de imágenes se calcula con la ecuación 3.2. Cuanto más oscuro es un color en la capa de fusión, más se oscurece el color similar en la capa base. Se consigue de este modo aumentar el contraste y la saturación de los colores de la capa base, dándoles un aspecto más intenso, como se ve en la figura 3.8.

$$C_{out} = 1 - \frac{1 - C_{base}}{1 - C_{blend}} \quad (3.2)$$

Donde:

- C_{out} es el valor del píxel resultante en el canal de color.
- C_{base} es el valor del píxel de la capa base en el canal de color.
- C_{blend} es el valor del píxel de la capa de mezcla en el canal de color.

Esta fórmula se aplica por canal de color (rojo, verde, azul) de forma independiente, para píxeles con valores normalizados en el rango [0,1] (con escalado Min-Max se divide por 255).

3.3.4. Imágenes de suelo

Se contaba con dos imágenes del suelo sin sembrar, provistas por el cliente. Estas fueron incorporadas al dataset con el objetivo de aportar mayor información sobre el fondo y ayudar al modelo a reconocer posibles sectores sin plantas o con excesiva cantidad de rastrojo.

Al detectarse que las imágenes contenían algún tipo de planta (posiblemente malezas o arroz guacho), se decidió aplicar una estrategia de censura para las regiones de la imagen que contenían píxeles verdes. Esto se implementó mediante el índice exceso de verde (*Excess Green*) con un umbral, para generar una máscara de píxeles negros que ocultaran las plantas, como se puede ver en la imagen 3.9.

Luego se generó un archivo .TXT de etiquetas completamente vacío, con el mismo nombre que las imágenes del suelo.

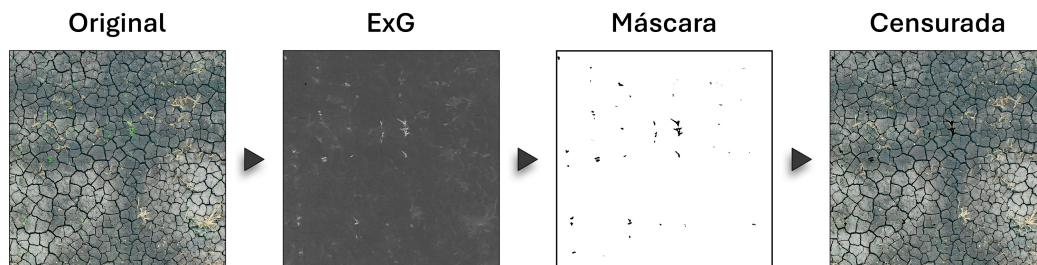


FIGURA 3.9. Procesamiento de imágenes del suelo.

3.4. Preprocesamiento de imágenes y etiquetas

El preprocesamiento de imágenes se aplica de modo similar tanto al *pipeline* de inferencia como al de entrenamiento, para asegurar que el proceso de ETL (*Extract, Transform, Load*) sea el mismo. La única excepción es el *script* de “división de etiquetas” que se ejecuta únicamente durante el entrenamiento. Para la inferencia, solo es necesario preprocesar la división de la imagen que alimentará el modelo.

3.4.1. División en mosaicos

Cada imagen de dron se dividió en mosaicos cuadrados (*tiles*) para presentar los datos en el formato aceptado por el modelo YOLOv8. Si bien esta versión del modelo soporta un rango de dimensiones posibles para las imágenes de entrada, estas siempre son escaladas a 640×640 px; por este motivo, se decidió trabajar con esas dimensiones en la división para evitar pérdida de información.

El *script* desarrollado permite procesar imágenes de cualquier tamaño y orientación, tan solo con indicar las dimensiones del mosaico de salida (siempre cuadrado), por defecto de 640 px de lado (nombrado como constante S de aquí en adelante).

La única limitante de tamaño está impuesta por el formato del nombre de archivo que permite indicar hasta 100×100 mosaicos (con valores de 00 a 99). Dicha restricción no es relevante para el caso de uso previsto.

Offset (opcional)

De modo opcional, puede agregarse un borde negro antes de la división para que se genere un número entero de mosaicos (todos de igual dimensión S). De este modo, la imagen es centrada automáticamente (como se aprecia en la figura 3.10) para que los n *tiles* de los bordes contengan la mayor cantidad de información posible. Si se desactivara, los mosaicos de los bordes derecho e inferior serían más pequeños por tener menos píxeles que la constante S .

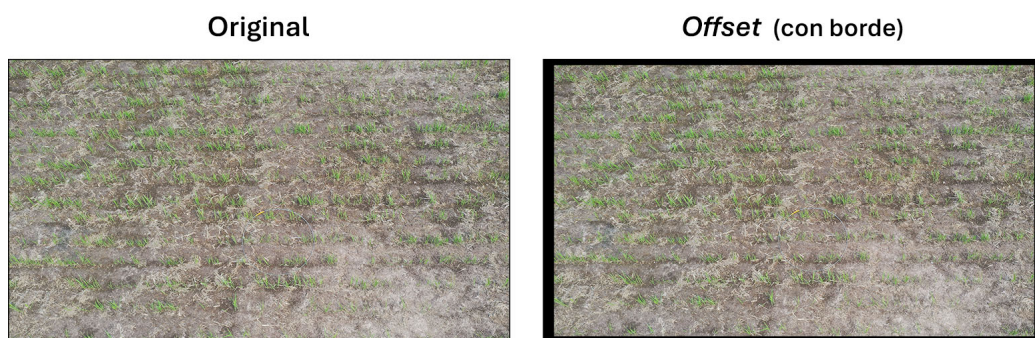


FIGURA 3.10. Imagen original y modificada con *offset*.

Si bien no es imprescindible para la inferencia, se recomienda mantener activada esta opción ya que el modelo fue entrenado con imágenes con estas características y podría afectar negativamente en la inferencia.

División de imágenes

Para la división de las imágenes, se importan las imágenes RGB con la biblioteca OpenCV. Se calcula la cantidad de filas y columnas que se generarán (alto y ancho, dividido S). Luego, se calculan las coordenadas (x_0, y_0, x_1, y_1) en las que se recortará la matriz de píxeles. Se logra al dividir las dimensiones de la imagen (incluido el borde negro) por el número de fila y columna correspondientes multiplicado por la constante S . Este proceso se itera para cada uno de los mosaicos a generar, almacenándolos en un *array* de listas (`tiles[row][column]`) para preservar el ordenamiento de los datos con su naturaleza matricial. Cada recorte es almacenado en disco con el formato de entrada (`.jpg`) con OpenCV.

División de etiquetas (solo para entrenamiento)

Este proceso solo se aplica para el *pipeline* de entrenamiento y permite distribuir aquellas etiquetas de la imagen etiquetada que pertenecen a mosaicos diferentes.

Dividir las etiquetas es un proceso crítico, ya que si se pierde la correlación espacial con la imagen original, el modelo no podrá aprender de las imágenes, ya que las etiquetas no tendrán sentido. Por este motivo, fueron implementadas múltiples etapas de verificación que aseguran que el número de etiquetas de entrada y salida sea consistente con el proceso realizado.

Para comenzar, se importan las etiquetas de la imagen completa en formato `.TXT` y se crea un *DataFrame* que contiene el de etiquetas importadas. Como se mencionó anteriormente, por seguir el formato YOLOv8 PyTorch TXT, las etiquetas se expresan en coordenadas relativas que indican la ubicación de cada centroide y las dimensiones del *bounding box*:

```
class_id  center_x  center_y  width  height
0          0.187222  0.873889  0.014168  0.010380
```

Es por esto que primero son convertidas en valores absolutos:

```
class_id  center_x  center_y  width  height
0          1024      2690      78      32
```

Es aquí cuando se incorpora el borde negro definido previamente para la imagen, lo que desplaza cada etiqueta en la dirección (x, y) del vector *offset*. A continuación se calculan las coordenadas de los puntos de inicio (x_0, y_0) y fin (x_1, y_1) para cada *bounding box*:

```
class_id  x0      x1      y0      y1
0          3443  3503  2499  2541
```

Todas estas transformaciones se realizaron de forma matricial para procesar todos los datos de la forma más eficiente.

Luego, se ingresa al bucle de división de etiquetas, que encuentra para cada *tile* todas aquellas etiquetas que le pertenecen y realiza la división del *bounding box* para etiquetas compartidas con mosaicos adyacentes.

Se generó una estructura de variables para almacenar toda la información de forma unificada, a través de una lista que almacena para cada mosaico un diccionario con la tupla de coordenadas del recorte (x_0, y_0, x_1, y_1) , el *DataFrame* con las etiquetas que le pertenecen (recortadas) y la matriz de píxeles del *tile*.

Mediante un minucioso proceso de depuración y testeo, se garantizó el funcionamiento óptimo de este proceso crítico.

Finalmente, se recorre nuevamente la matriz de mosaicos para recuperar los datos almacenados en la estructura de variables, convertir nuevamente los *bounding boxes* a etiquetas y pasar de valores absolutos a relativos. Se agrupan todas las etiquetas en una misma línea de texto y se almacenan en el nuevo archivo `.TXT` generado para el mosaico.

Dado que se trata de un proceso con múltiples bucles anidados, suele demorar a cerca de un minuto en completar la ejecución. Por este motivo, se incorporó una barra de progreso y un *output* detallado (*verbose*) para indicar al usuario el nivel de avance.

Gestión de nombres de archivos

Para poder mantener una coherencia en la gestión de archivos, se decidió agregar el identificador del mosaico antes del código de Roboflow, es decir, junto al nombre de la imagen original.

Se adoptó el siguiente formato: `tile`, número de fila, `x`, número de columna.

A continuación se muestra un ejemplo:

- Entrada:
`209_101_24_JPG.rf.8d87d842911d...37ccc299d413`
- Salida:
`209_101_24_JPG_tile01x06.rf.8d87d842911d...37ccc299d413`

Se le agregó la extensión correspondiente (`.jpg` o `.txt`) según se tratase del mosaico o sus etiquetas.

Para el *pipeline* de inferencia, cada uno de los mosaicos generados es almacenado en una carpeta temporal de forma local (`data/inference/nombre_de_imagen`), que es eliminada de forma automática una vez finalizado el conteo de plantas.

3.5. Ecualización adaptativa (CLAHE)

Se decidió aplicar ecualización adaptativa a cada mosaico para limitar el riesgo de variabilidad de las imágenes capturadas en entornos productivos, donde el brillo, contraste, exposición o saturación pueden variar ampliamente según las condiciones climáticas, la hora del día y la estación del año.

Para cada *tile* se calcula el histograma de intensidad, se aplica el límite de contraste (*clipping*) para evitar la amplificación excesiva del ruido y se redistribuyen los valores que superen ese umbral a otros *bins* del histograma. Finalmente, se calcula una función de transformación basada en el histograma modificado y se aplica a los píxeles dentro del *tile* para ajustar su contraste.

Se probaron distintos valores de configuración y se eligieron los que ayudaban a diferenciar mejor las plantas del fondo: límite 2 (`clip limit`) y tamaño de cuadrícula 16×16 (`grid size format`).

3.6. Normalización (escalado de rango)

YOLOv8 aplica de forma automática normalización por escalado, donde los valores de los píxeles se escalan al rango de $[0, 1]$ dividiéndolos por 255. Esta técnica asume los valores mínimo (0) y máximo (255) del rango original de forma constante para todas las imágenes. Es por eso que también se le conoce como normalización $[0, 1]$ o escalado de 0 a 1.

3.7. Preprocesamiento para falso color (opcional)

En el caso de trabajar con el dataset de falso color (FC), se agrega una etapa inicial de preprocesamiento antes de la división en mosaicos para transformar las imágenes de RGB al nuevo espacio de color. El resto del proceso *pipeline* continúa inmutado.

Como se puede ver en la figura 3.11, se aplican secuencialmente las transformaciones antes mencionadas:

1. *Excess Green* (Escala de grises)
2. Reducción dimensional: RGB $\rightarrow$ 2-PCA (Escala de grises)
3. Combinación: 2-PCA + ExG (Falso color)
4. Fusión Color Burn: Falso color + RGB (Falso color)

De este modo se genera un dataset alternativo que logra destacar las plantas y opacar el fondo al combinar las ventajas que aporta el índice exceso de verde, sin pérdida de información (por la aplicación de PCA) y sin pérdida de resolución en la imagen (por la fusión con *color burn*).

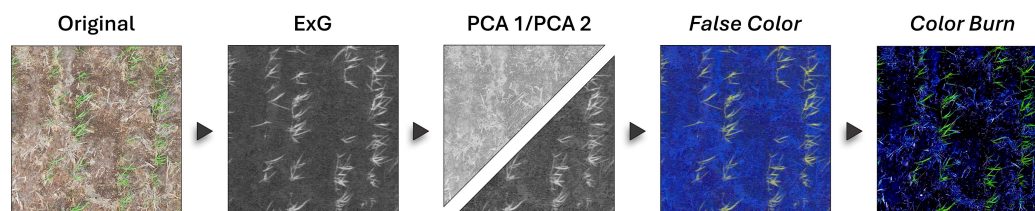


FIGURA 3.11. Proceso de generación de imágenes de falso color.

3.8. Modelo de detección de objetos

En esta etapa se realiza la inferencia para cada uno de los mosaicos almacenados en `data/inference/nombre_de_imagen`.

Se entrenaron un total de 90 modelos diferentes con un tiempo total estimado de uso de GPU de 270 horas, para una utilización de VRAM de entre el 75 % y el 95 %. Puede consultarse la tabla D.1 para ver en detalle el hardware utilizado.

3.8.1. Split de datos

El dataset final está formado por un total de 896 mosaicos generados a partir de 14 imágenes cenitales de alta resolución de los cultivos capturadas a una altura de 3,5 m y 2 imágenes de suelo sin sembrar. Se trabajó con una división de 57,8 %

para los datos para entrenamiento, 24,1 % para validación y 18,1 % para testeo. Al incluir *Data Augmentation* $9\times$, se llegó a un total de 3542 mosaicos para el conjunto de entrenamiento.

3.8.2. Entrenamiento de modelos

Todos los experimentos para diferentes modelos fueron entrenados a través de la API de Ultralytics (importando la biblioteca del mismo nombre) [45]. Dado que los entrenamientos se realizaron en la nube, se vincula el *notebook* con una carpeta compartida en Google Drive que contiene las distintas versiones del dataset que se usaron para los experimentos, como se puede ver en la figura 3.12. Los datos a utilizar son copiados a la máquina virtual para poder acceder a ellos de forma eficiente.

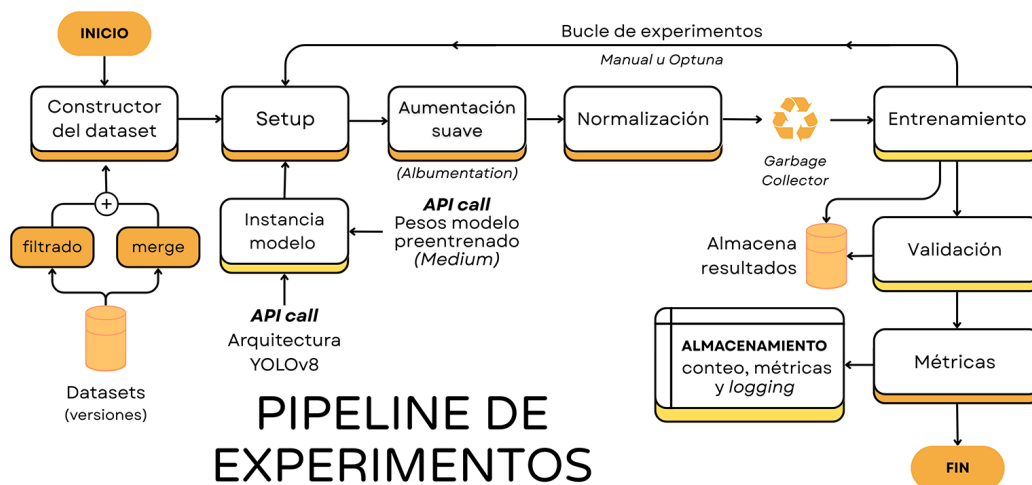


FIGURA 3.12. Diagrama de *scripts* para experimentación.

Luego, se importa la biblioteca YOLO que define la arquitectura del modelo y se le cargan los pesos pre-entrenados para la versión correspondiente (mayormente YOLOv8 Medium) descargando de Ultralytics el archivo PyTorch (`yolo_v8m.pt`). Para algunos experimentos el modelo fue inicializado con pesos aleatorios a través del archivo YAML (`yolo_v8m.yaml`).

Antes de iniciar el entrenamiento se ejecuta una instancia de *Garbage Collection* (`gc.collect()`) para limpiar la memoria, ya que los recursos en Colab son limitados y es común la interrupción de la sesión si no se liberan recursos no utilizados. También se establece la variable de entorno `PYTORCH_CUDA_ALLOC_CONF` con `expandable_segments:True`, para intentar reducir la fragmentación de la VRAM y así utilizar la GPU de forma más eficiente.

Se inicia así el entrenamiento del modelo en cuestión con la configuración de hiperparámetros que se desea explorar. Para la mayoría de los entrenamientos, se estableció una limitación por tiempo de entrenamiento según los recursos disponibles y se definió el máximo tamaño de lote (`batch size`) para aprovechar al máximo la GPU.

La complejidad de esta arquitectura es un tamaño a considerar, ya que, para el tamaño *Medium*, se ajustan aproximadamente 25,9 millones de parámetros por cada época. Incluso a pesar de haber aplicado la técnica de *Transfer Learning*, fue necesario realizar entrenamientos largos para darle tiempo al modelo de que aprenda

las características de las nuevas imágenes, dado que la arquitectura YOLO fue pre-entrenada para un dataset (COCO) con un dominio visual bastante diferente. Un entrenamiento estándar para los datasets desarrollados demoraba entre 2 y 4 horas para alcanzar su mejor época.

Por este motivo, se decidió darle al modelo la posibilidad de realizar entrenamientos largos: se limitó el número de épocas (de 500 a 1000) o el tiempo de ejecución (1 a 4 h); pero siempre con *Early Stopping* acorde a los objetivos del experimento, para que el entrenamiento se interrumpiera de no haber mejoras considerables.

Una vez finalizado el entrenamiento, es crucial realizar inmediatamente una copia de respaldo en la nube, ya que, de interrumpirse la sesión, se pierde todo el trabajo realizado. Para mitigar este riesgo, se implementó un *script* que almacena de forma automática en Drive todos los resultados del entrenamiento (pesos y gráficos).

3.8.3. Validación de modelos

Para el proceso de validación, se cargan a la arquitectura YOLO los pesos del nuevo modelo entrenado (`weights/best.pt`), esto puede verse en la figura 3.13. Al igual que para el entrenamiento, se utilizó el mayor *batch size* soportado por la GPU. Se activaron las funciones `verbose` y `save_json` para almacenar la mayor cantidad de información posible sobre cada entrenamiento.

Se generó un *script* para extraer la matriz de confusión de los resultados entregados por YOLO y calcular así las métricas de interés: *Accuracy*, *Precision*, *Recall*, *F1-Score*, *F1/2-Score* y *G-measure* (que serán explicadas en detalle en el siguiente capítulo).

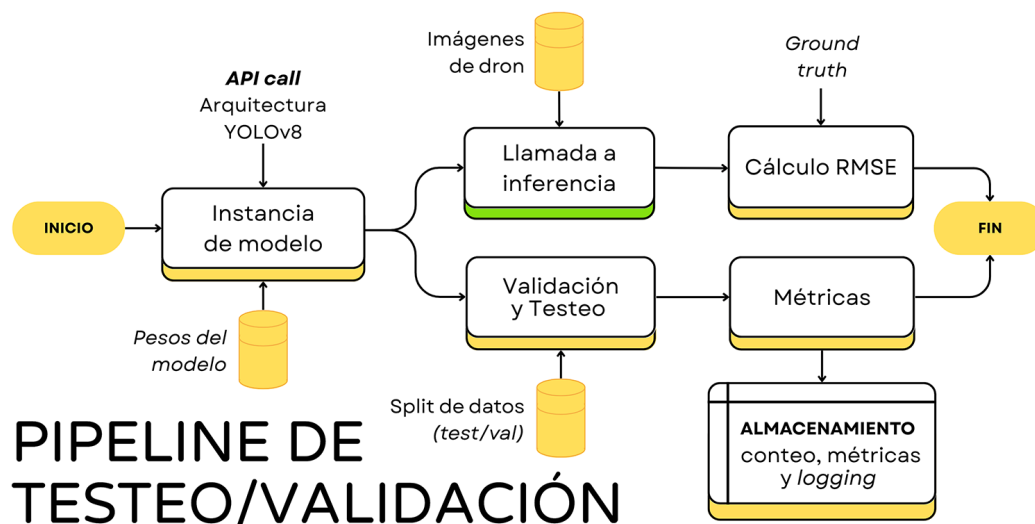


FIGURA 3.13. Diagrama de *scripts* para validación y testeo.

De forma complementaria, se realizaron pruebas de validación en las que se definieron manualmente distintos hiperparámetros o se exploraron a través de búsquedas estocásticas con Optuna.

3.8.4. Inferencia: módulo de detección de plantas

Para el *pipeline* de inferencia, mostrado en la figura 3.1, se instancia un modelo YOLO y se le cargan los pesos del mejor modelo entrenado (o el modelo en cuestión que se desee testear). Se importan todos los mosaicos generados durante la división de la imagen (almacenados en: `data/inference/nombre_de_imagen`).

De forma secuencial se realiza la inferencia para cada mosaico y se almacenan los *bounding boxes* de los objetos detectados por el modelo en varios formatos. En primer lugar, se guarda el archivo `.TXT` con las predicciones en formato YOLOv8 PyTorch (contiene clase y coordenadas de cada objeto).

Además, por cada inferencia se almacena la visualización con *bounding boxes* generada por YOLO (esta opción se desactiva en producción) y un archivo `.JSON` que contiene información detallada para depuración:

- Nombre de la imagen.
- Nombre de clases posibles.
- Cada una de las detecciones con:
 - Coordenadas de la caja (x_0, y_0, x_1, y_1).
 - Confianza.
 - Clase predicha.
- Dimensiones de la imagen.
- Velocidad (para el modelo YOLO):
 - Preprocesamiento.
 - Inferencia.
 - Posprocesamiento.

Estos archivos se almacenan bajo el nombre original antecedido por el prefijo `"inference."` en el mismo directorio (`data/inference/nombre_de_imagen`).

Para la etapa de *testing*, se creó un cuaderno Jupyter que facilita la iteración sobre múltiples modelos que se deseen comparar. En este se realizan inferencias con cada uno para una misma imagen, se guardan las predicciones, métricas y archivos de registro (*logs*) para la depuración. Este *notebook* presenta las múltiples predicciones, mide el tiempo de inferencia para cada modelo e indica el porcentaje de progreso del proceso. Al finalizar, presenta el tiempo total de inferencia para una imagen completa (todos los mosaicos) y tiempo medio de inferencia/mosaico.

3.9. Posprocesamiento de predicciones

Finalmente, se unifican los mosaicos y predicciones (almacenados dentro de `data/inference/nombre_de_imagen`) para generar el conteo para la imagen completa y calcular así la densidad de plantas por hectárea.

3.9.1. Reconstrucción y limpieza de etiquetas

Se realiza el proceso inverso al de división de etiquetas con el objetivo de unificar todas las detecciones realizadas durante las múltiples inferencias. Primero, se reconstruye la grilla de mosaicos para iterar sobre la matriz de *tiles*, se asignan a cada inferencia las coordenadas correspondientes en la imagen completa (incluye el *offset* del borde negro).

Se abre cada uno de los archivos .TXT con las predicciones para el mosaico correspondiente y se las importa a un *DataFrame*. Este se almacena dentro de un diccionario junto con la posición de la matriz de mosaicos (fila/columna).

Para cada *tile*, se transforman las etiquetas de valores relativos a absolutos y se calculan las coordenadas de los *bounding boxes* (punto inicial y final de la caja). Se crea un nuevo *DataFrame* vacío en el que se concatenan todas las detecciones halladas para cada mosaico. Por ser un proceso crítico, se implementaron diversos pasos de verificación que garantizan que no haya pérdidas de datos durante el proceso.

Una vez unificados todos los *bounding boxes*, se los vuelve a convertir en etiquetas (punto central, ancho y alto) y transformar a valores relativos, para completar la exportación al archivo .TXT final.

3.9.2. Detección de líneas de cultivo

Dadas las características de los campos del cliente, se parte de la premisa de que todos los surcos presentan una distribución prácticamente paralela. El sistema no está pensado para analizar patrones en surcos curvilíneos u orientaciones variables (líneas oblicuas).

Para comenzar, se estima la pendiente general de las líneas de cultivo mediante ajuste polinómico de mínimos cuadrados (con `polyfit`), en este caso para polinomios de primer grado por tratarse de rectas. Se ajusta una recta $f(x) = mx + b$ al conjunto general de puntos y se calcula el ángulo de inclinación $\theta = \arctan(m)$.

Luego se aplica una matriz de rotación (3.3) para lograr que las líneas de cultivo se alineen horizontalmente (condición necesaria para el adecuado funcionamiento de DBSCAN). Como se ve en la ecuación 3.4, se multiplica al vector de posición $\vec{p}_{(x,y)}$ por la matriz.

$$R(-\theta) = \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{bmatrix} \quad (3.3)$$

$$\vec{p}_{rotado} = R(-\theta) \cdot \vec{p}_{(x,y)} \quad (3.4)$$

A continuación se aplica el algoritmo de *clustering* DBSCAN, con una tolerancia de 0,01 de distancia y un mínimo de 5 puntos por *cluster*. Este reúne todos aquellos puntos que podrían pertenecer a un mismo grupo asignándoles un índice (entero positivo) y marca los puntos que no pudieron ser asignados con valor -1.

Si bien DBSCAN podría identificar *clusters* con cualquier orientación, se obtienen resultados más precisos al alinear todos los puntos con un único eje, ya que se simplifica la búsqueda de vecinos a una sola dimensión. Esto lo vuelve más eficiente y efectivo, como puede apreciarse en la figura 3.14.

FIGURA 3.14. Asignación de *clusters* realizada por DBSCAN.

Posteriormente se utiliza nuevamente `polyfit` para encontrar la recta que mejor se ajusta a cada *cluster* y se crea un diccionario con los coeficientes para cada grupo.

En esta instancia, se verifican todas aquellas rectas que se encuentren superpuestas dentro de cierto margen de tolerancia y se reasignan sus puntos a un mismo clúster (por ser falsos positivos en la asignación de DBSCAN).

Por último, se mide la distancia a la recta para todos aquellos puntos a los que se los pudo agrupar y se los asigna de forma tentativa al *cluster* más cercano. En la figura 3.15 puede apreciarse el resultado de este proceso.

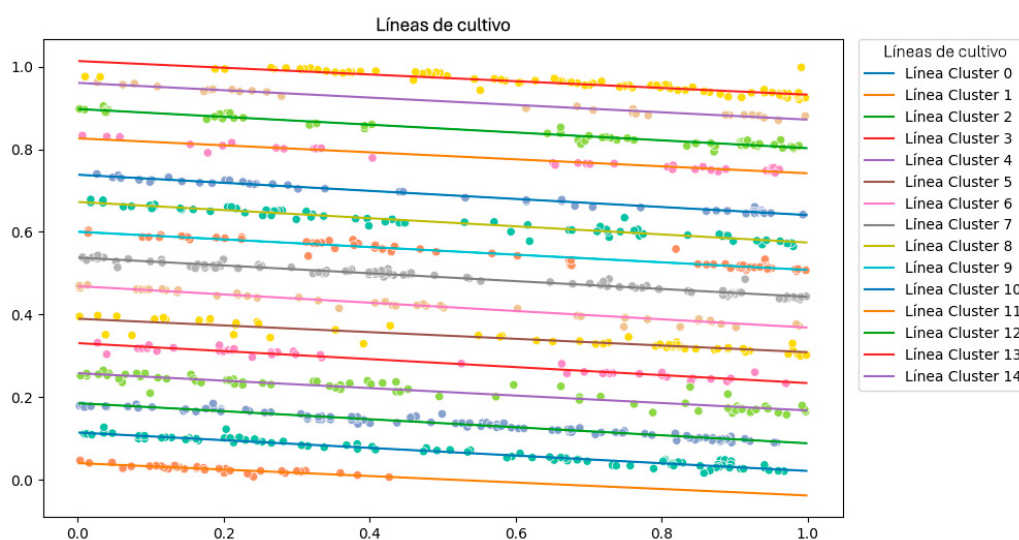


FIGURA 3.15. Detección de líneas de cultivo.

3.9.3. Discriminación de malezas

Se considera “maleza” a todos aquellos puntos que se encuentran posicionados con una distancia superior al umbral en dirección perpendicular de la línea de cultivo (ya se trate de arroz rojo, arroz guacho u otras especies invasoras). De forma empírica se determinó que con un umbral de 4 desviaciones estándar se consigue filtrar aquellas plantas que se encuentran entre surcos, sin perder detecciones correctas por la elevada dispersión. Cada punto que supere este umbral es marcado como “potencial outlier”.

A continuación, los posibles *outliers* son asignados al grupo inmediato adyacente para verificar si fueron erróneamente asignados a un *cluster* vecino y se verifica la distancia a la nueva recta. Solo aquellos objetos que fueran identificados como *outliers* para ambas líneas serán efectivamente “maleza”. Puede observarse un ejemplo de las detecciones de malezas en la figura 3.16.

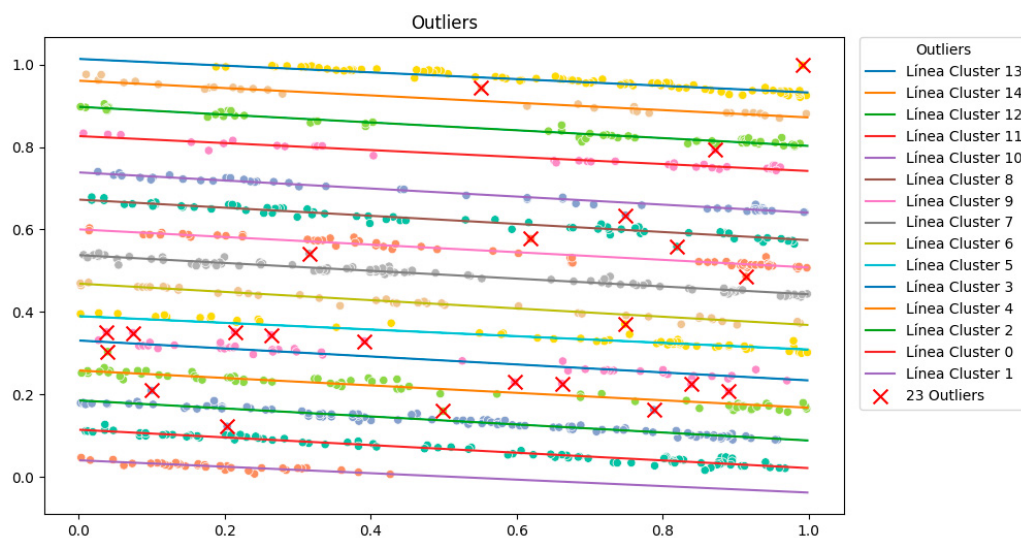


FIGURA 3.16. Detección de malezas (*outliers*).

Es importante destacar que cada punto indica el centroide del *bounding box* (no el tallo de la planta), por lo que esta dispersión se debe generalmente a que la planta se encuentra recostada sobre el suelo. Este aspecto se agrava cuanto mayor es el tamaño de la planta, ya que deteriora progresivamente la capacidad del modelo para detectar malezas.

3.9.4. Conteo de plantas y cálculo de densidad

Finalmente, se realiza un conteo de aquellos puntos que no fueron marcados como “maleza”. Se verifica además que no se hayan perdido objetos durante el proceso de filtrado.

El área cubierta por la imagen en función de la altura es un parámetro conocido, que fue medido de forma experimental durante la construcción del dataset. En la tabla 3.6 pueden verse los valores de referencia considerados para las alturas utilizadas en los experimentos. En caso de requerirse, por trabajar en otra altura o con diferente resolución, es posible que el usuario introduzca un valor personalizado para el cálculo del área.

TABLA 3.6. Valores del área cubierta por la fotografía.

Altura (m)	Resolución (cm/px)	Área (m ²)
3,5	0,082	13,35
5	0,120	28,75

Dada esta constante, se calcula la densidad de plantas por m² y por hectárea, con la ecuación 3.5:

$$densidad_{ha} = \frac{n_{plantas}}{área} \times 10\,000 \quad [pl/ha] \quad (3.5)$$

Ambos valores de densidad son entregados por consola al usuario, junto con el conteo y el número de malezas. Por cada imagen procesada, se genera un archivo de *log* y se almacenan los resultados del conteo en un archivo CSV.

3.10. Procesamiento por lote

Para procesamiento en *batch* se repite secuencialmente el proceso de conteo para cada muestra (n imágenes de un mismo lote). Como se explicó anteriormente, el resultado de cada conteo individual es almacenado en disco. Luego, se importan los datos del CSV correspondientes a las imágenes procesadas y se calcula la media aritmética (valor medio) de la densidad por hectárea con la ecuación 3.6:

$$\bar{D}_{ha} = \frac{1}{n} \sum_{i=1}^n d_{i,ha} \quad (3.6)$$

3.11. Batería de métricas

En las reuniones con el cliente se informó que una medición que sobreestimara la densidad de plantas implicaba un riesgo mayor para la empresa, dado que impedía que se tomaran medidas paliativas durante el cultivo, detectándose recién el problema a la hora de la cosecha, lo que les genera una pérdida monetaria por disminución del rinde.

Por el contrario, una medición sesgada a la baja generaría una mayor atención para verificar la situación y accionar en consecuencia. Se determinó entonces que era prioritario limitar los Falsos Positivos (FP) por sobre los Falsos Negativos (FN).

Por este motivo, en un principio se decidió escoger *Precision* como indicador clave de rendimiento (KPI) para cuantificar la proporción entre Verdaderos Positivos (TP) y Falsos Positivos (FP), como se puede ver en la ecuación 3.7:

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (3.7)$$

Posteriormente, se decidió construir una métrica personalizada que se ajustara mejor a la naturaleza del problema. Se optó por trabajar con F_{β} -score, con la ecuación 3.8, y asignarle un valor $\beta = \frac{1}{2}$ para darle mayor peso a la *Precision* que al *Recall*, y así intentar reducir los FP. La métrica final puede encontrarse en la

ecuación 3.9.

$$F_{\beta} = (1 + \beta^2) \frac{\text{Precisión} \times \text{Recall}}{(\beta^2 \times \text{Precisión}) + \text{Recall}} \quad (3.8)$$

Sustituyendo $\beta = \frac{1}{2}$:

$$F_{1/2} = \left(1 + \left(\frac{1}{2}\right)^2\right) \frac{\text{Precisión} \times \text{Recall}}{\left(\left(\frac{1}{2}\right)^2 \times \text{Precisión}\right) + \text{Recall}} = \frac{\frac{5}{4} \text{Precisión} \times \text{Recall}}{\frac{\text{Precisión}}{4} + \text{Recall}} \quad (3.9)$$

De forma complementaria se incorporó el *índice Fowlkes-Mallows (G-measure)*, de la ecuación 3.10, que es la media geométrica entre *Precision* y *Recall*.

$$\text{G-measure} = \sqrt{\text{Precisión} \times \text{Recall}} \quad (3.10)$$

A diferencia de F_{β} (media armónica), que penaliza fuertemente los valores bajos, la media geométrica es más permisiva con los valores extremos. Un modelo que se enfoque en lograr una precisión elevada —para minimizar los falsos positivos— podría tener un *recall* ligeramente menor, que sería penalizado por una métrica como *F1-score*, incluso aunque representara una mejora dentro de ese contexto. *G-measure* es útil cuando se quiere mantener un equilibrio entre ambos valores, sin llegar a una precisión excelente a costa de detectar muy pocos objetos reales.

Además, se utilizó *Mean Average Precision* (mAP) como métrica de evaluación para tareas de detección de objetos. A diferencia de la precisión simple, que evalúa la exactitud de las predicciones en general, el mAP considera tanto la precisión como el *recall* a diferentes niveles de confianza o umbrales, y luego promedia estos valores, como puede verse en la ecuación 3.11. Dado que determinar la posición exacta de los *bounding boxes* no era algo determinante, ya que lo importante era la detección en sí misma, se utilizó el mAP para un umbral de Intersección sobre Unión (IoU) del 50 % (denominado en el contexto de YOLO como mAP@0.5).

$$mAP = \frac{1}{k} \sum_i^k AP_i \quad (3.11)$$

A modo complementario, se utilizaron también las métricas tradicionales (*Accuracy*, *Recall* y *F1-score*) como referencia durante las pruebas, pero no fueron consideradas para la toma de decisiones críticas de desarrollo.

Capítulo 4

Ensayos y resultados

En este capítulo se presentan los resultados obtenidos para los diferentes experimentos realizados durante el desarrollo del trabajo.

4.1. Ensayos con modelos

En total, se llevaron a cabo 35 experimentos con diferentes configuraciones de hiperparámetros y conjuntos de datos evaluados. Se entrenaron un total de 90 modelos, para los que se realizaron 460 pruebas de validación/testeo y 2214 inferencias para procesar mediciones sobre 42 imágenes con diferentes características.

Hasta el experimento 15 se trabajó con mosaicos de 240×240 px. A partir de entonces, se cambió la resolución a 640×640 px por ser el formato de entrada solicitado por YOLOv8.

Los experimentos realizados pueden agruparse en:

- Construcción del dataset.
- Pruebas con *Data Augmentation*.
- Experimentos con índices de vegetación (falso color).
- Arquitectura YOLO:
 - Versiones: v8, v9, v11, v12.
 - Tamaños: *Nano / Medium / Large / Extra-large*.
- *Fine-Tuning (freezing)*:
 - Inicialización aleatoria (sin preentrenamiento)
 - *Fine-Tuning* completo: modelo preentrenado sin congelado,
 - Congelando parte del *Backbone* (8 capas),
 - Congelando todo el *Backbone* (10 capas),
 - Congelando parte del *Head* (12 capas).
- Experimentos con hiperparámetros: *batch size*, número de épocas, *early stopping*, tasa de aprendizaje, *momentum*, *multi-scale training*, regularización *dropout* y L2 (*weight decay*, decaimiento de pesos) y ganancia de la función de pérdida (*box loss*, *classification loss* y *distribution focal loss*).

- Pruebas de validación: confianza (*confidence*), IOU y NMS agnóstico (*agnostic non-maximum suppression*).
- Experimentos de inferencia (*testing*).

El detalle del hardware utilizado puede ser encontrado en el apéndice [D](#).

4.1.1. Pruebas con diferentes arquitecturas

Si bien se realizaron pruebas de diseño con otras versiones de YOLO (v9, v11 y v12) se decidió trabajar con la versión v8 por ser la que ofreció mejor balance entre calidad de detecciones y tiempo de entrenamiento. Las versiones más recientes están optimizadas para procesamiento de videos al requerir menor tiempo de inferencia. Esto no representaba una ventaja adicional para el problema aquí expuesto.

Se observó que las versiones más modernas de YOLO lograron un mAP más alto, pero incrementando considerablemente los FP. Estos modelos mejoran en la detección espacial con los *bounding boxes*, pero deterioran las métricas *Precision* y *F_{1/2}-score*. Por estas razones se decidió utilizar YOLOv8 como arquitectura base.

Además, se realizaron experimentos para los distintos tamaños de la red (*Nano*, *Medium*, *Large*, *Extra Large*). El cambio de *Nano* a *Medium* representó una considerable mejora (+25 % en *Precision* y para *F_{1/2}-score* de +17 %).

Sin embargo, para los modelos de mayor escala se obtuvieron mejoras en esas métricas (para *Extra*, *Precision* +15 % y *F_{1/2}-score* +10 %), pero con incrementos considerables de los tiempos de entrenamiento (4,6x) e inferencia (2,7x) y de los recursos de hardware necesarios (tanto en entrenamiento como inferencia); para lograr apenas una mejora de +4 % en los TP. Por estos motivos, no se consideró un trade-off oportuno utilizar modelos mayores a *Medium* durante el desarrollo.

4.2. Construcción del Dataset

Se realizaron experimentos preliminares con pequeños conjuntos de imágenes etiquetadas a alturas de 3,5 m y 5 m, con ángulos de 90° y 60° (con plantas en desarrollo temprano, medio y avanzado). Se realizaron entrenamientos piloto con estos pequeños datasets, para comparar el impacto de los distintos tipos de datos en el desempeño del modelo desde una etapa temprana.

Durante el proceso de etiquetado se colocaron un total de 49 285 *bounding boxes* dentro de las cuatro clases mencionadas en el capítulo anterior. Se puede ver en la tabla [4.1](#) que existe una relación de aproximadamente 6:1 entre las clases `plant-weed` (etiqueta de alta confianza) y `plant-occluded` (etiqueta de baja confianza). Esta última corresponde a potenciales casos problemáticos que pudieran ser difíciles de detectar para el modelo: plantas superpuestas, ocluidas o extremadamente pequeñas.

En la figura [4.1](#) se observa que las etiquetas son de forma y tamaños similares, con distribución aleatoria en el plano (x, y) y sin encontrarse sesgadas.

Es importante recordar que esta diferenciación de clases solo se utilizó para el etiquetado, por la gran complejidad de las imágenes que contenían una media de

2 mil plantas. Al momento del entrenamiento, las únicas clases utilizadas fueron `plant-weed` y `plant-occluded`, unificadas bajo una misma clase.

TABLA 4.1. Distribución de etiquetas por clase.

Class	Totales	
intruder	30	0%
plant-cluster	715	1%
plant-occluded	6647	13%
plant-weed	41 878	85%
Total etiquetas	49 285	
Plantas/img	2022	

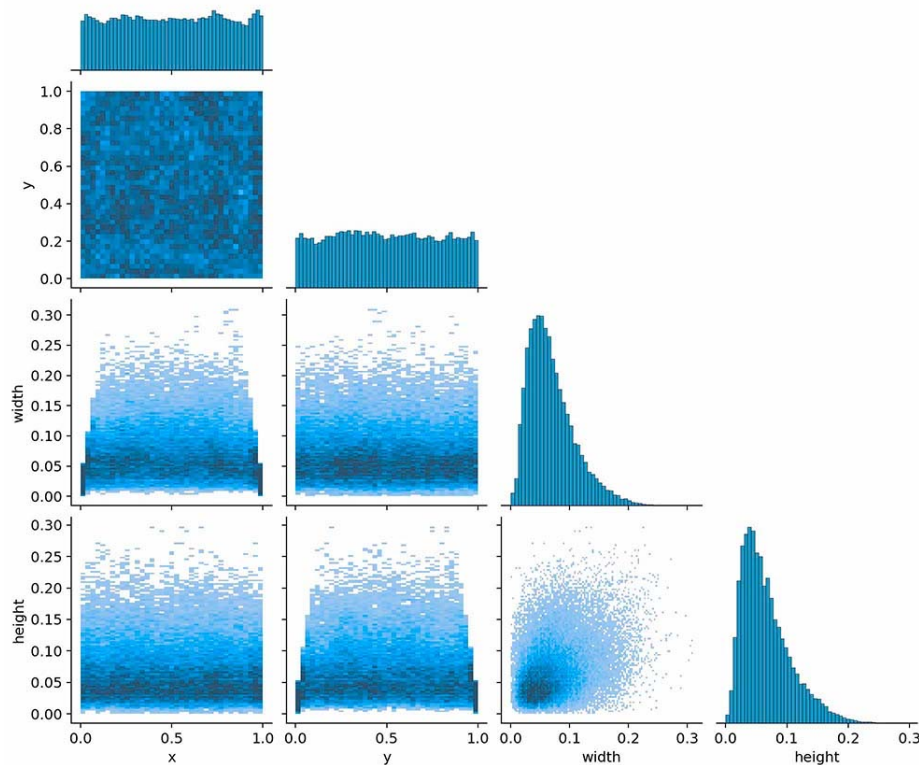


FIGURA 4.1. Correlograma de etiquetas.

4.2.1. Resultados con imágenes anguladas

Se observó que era notablemente más sencillo el etiquetado de las imágenes con 60° de inclinación, ya que permiten reconocer más fácilmente el tallo de la planta. Sin embargo, las pruebas de entrenamiento mostraron un incremento de los falsos positivos (+10%) respecto a las imágenes a 90° ; esto perjudica la detección de verdaderos positivos en similares proporciones.

Las imágenes con ángulos de 60° no fueron incorporadas a la versión final del dataset ya que no permitían realizar una medición precisa de la densidad de plantas.

4.2.2. Resultados para distintas alturas de vuelo

Se verificó la observación inicial, hecha durante la clasificación de los datos. Al aumentar la altura de vuelo, si bien aumenta la superficie cubierta, lo que permite estudiar una porción mayor del cultivo, también disminuye la resolución de la cámara (medida en cm por píxel). Esto produce que el incremento de cobertura no llegue a compensar la pérdida de detalles (ver tabla 3.6).

Dado que los cultivos de arroz con los que se trabajó presentan dimensiones pequeñas en etapas tempranas de post-emergencia (con plántulas de entre 1 y 5 cm de altura), esto implica que se haga cada vez más difícil para el modelo reconocer objetos pequeños al incrementar la altura de vuelo.

Si bien para etapas tardías del desarrollo podría trabajarse a mayor altura, la planta de arroz tiene la característica de comenzar el macollaje entre la tercera y quinta semana de siembra. Esto significa que aumenta la densidad de la planta al brotar tallos laterales (macollos), lo que vuelve imposible determinar con exactitud el número de plantas.

Esto fue notorio durante el proceso de etiquetado de las imágenes, ya que se volvía extremadamente difícil determinar cuándo era una única planta y cuándo eran varias. Las pruebas iniciales de entrenamiento dieron resultados desfavorables que evidenciaron la incapacidad del modelo para reconocer objetos pequeños desde una gran altitud. Con plantas pequeñas, tuvo 85,6 % de falsos negativos. Para plantas en desarrollo avanzado, logró tan solo 7,8 % de verdaderos positivos, 24,2 % de falsos positivos y 68 % de falsos negativos.

Como se mencionó anteriormente, a través de la experimentación con los modelos se verificó que la ventana temporal óptima para realizar el conteo oscila entre una y tres semanas, antes de que la planta comience el macollaje.

4.3. Configuración de YOLOv8

Durante el desarrollo del *pipeline* de inferencia, se experimentó con distintas configuraciones de hiperparámetros para el modelo YOLOv8 y estrategias para aplicar las técnicas de *transfer learning* y *data augmentation*.

4.3.1. Transfer learning

Para la transferencia de aprendizaje (*Transfer learning*), si bien se exploró el congelamiento (*freezing*) de distinto número de capas de la arquitectura (incluyendo *backbone* y *head*), finalmente no fue aplicado para los modelos definitivos ya que se observaba una mejora al entrenar la red completa. Esto era algo esperable, por tratarse de imágenes de un dominio lejano al del dataset original (COCO).

Como se puede ver en la tabla 4.2, el mejor balance entre resultados y tiempo de entrenamiento se logró al cargar los pesos del modelo preentrenado de tamaño mediano (*Medium*) y luego continuar el entrenamiento de toda la red (detallado como «*Full Fine-Tuning (no freezing)*»).

Si bien el modelo inicializado aleatoriamente («*No pre-train (random weights)*») logró una precisión levemente mayor, al reducir los FP (-12,3 %), también fue capaz de reconocer menos TP (-5,6 %) que el modelo preentrenado. Además, requirió de un tiempo de entrenamiento 1,54x veces mayor.

TABLA 4.2. Resultados para *Transfer learning*.

Experimento	Precision	Recall	F $\frac{1}{2}$ -score	mAP@0.5
No pre-train (random weights)	59,8 %	54,0 %	0,587	0,449
Freezing Backbone (8 layers)	58,8 %	51,3 %	0,570	0,433
Freezing Backbone (10 layers)	53,7 %	57,0 %	0,543	0,451
Freezing Head (12 layers)	56,7 %	52,3 %	0,557	0,450
Full Fine-Tuning (no freezing)	58,1 %	60,3 %	0,586	0,495

En el caso del dataset de falso color, la utilización del modelo preentrenado impactaba negativamente en las predicciones. Por este motivo se trabajó con la inicialización aleatoria (cargando el archivo `YAML`). Es notable con este ejemplo cómo, al trabajar con imágenes de colores y formas notablemente diferentes al dataset de preentrenamiento, el modelo fue incapaz de trasladar el aprendizaje al nuevo dominio.

4.3.2. Ajuste de hiperparámetros

A continuación se describe la configuración utilizada para los modelos finales.

Función de pérdida ponderada (*Loss function*)

YOLOv8 utiliza una función de pérdida compleja que combina una ponderación entre la detección de las clases (`cls`), las coordenadas de los *bounding boxes* (`box`) y *Distribution Focal Loss* (`dfl`). La exploración de hiperparámetros arrojó los mejores resultados para una relación aproximada de 2 a 1 entre los valores de pérdida de clase y los otros dos (por ejemplo: `cls=3 | box=1,5 | dfl=1,5`). Esto se consiguió con la siguiente configuración de los ponderadores:

- `cls=1`, mayor al valor por defecto (0,5).
- `box=5`, menor al valor por defecto (7,5).
- `dfl=1,5`, valor por defecto.

Entrenamiento multiescala (`multi_scale`)

Este fue el hiperparámetro que mayor impacto positivo generó para todas las métricas de estudio. El entrenamiento multiescala ayuda a detectar objetos de diferentes dimensiones, al variar de forma aleatoria el tamaño de las imágenes de entrada. Como resultado, se obtiene una mejora en la capacidad de generalización del modelo.

Optimizador SGD

Dada la versatilidad de esta implementación de YOLO y la gran cantidad de hiperparámetros, se decidió utilizar la configuración automática para el optimizador, sugerida por el *framework* en función dataset.

En concreto, se utilizó el descenso de gradiente estocástico (SGD) como optimizador de la tasa de aprendizaje, con la siguiente configuración: `optimizer: SGD(lr=0,01, momentum=0,9)`.

Momentum

El *momentum* es un parámetro utilizado en el optimizador para acelerar el descenso de gradiente y evitar quedar atrapado en mínimos locales. Si bien se intentó configurarlo manualmente, YOLO lo fijó en todos los casos en 0,9 al usar el optimizador SGD.

Regularización L2 (*weight_decay*)

El *weight decay* es una técnica de regularización que penaliza los pesos grandes en el modelo, lo que ayuda a prevenir el *overfitting*. Se definió con un valor de 0,0015.

YOLOv8 establece de forma automática en qué grupos aplicar regularización L2:

- A 77 grupos de pesos no se les aplicó *weight decay* ($\text{decay} = 0$).
- En 84 grupos de pesos se les aplicó el valor establecido ($\text{decay} = 0,0015$).
- Para los 83 grupos de *bias* (sesgos) no es habitual aplicar regularización ($\text{decay} = 0,0$).

Dropout

Se realizaron experimentos con distintos niveles de *dropout* y se determinó que la incorporación de una pequeña cantidad de “apagado” de neuronas durante el entrenamiento ofrecía una pequeña mejora en las métricas. Esto ayuda a evitar que el modelo se vuelva demasiado dependiente de neuronas específicas y mejora su capacidad de generalización.

Los mejores resultados se obtuvieron con un rango menor al 10 %. Para el modelo definitivo se estableció en 5 %.

Otros hiperparámetros explorados

La implementación de YOLOv8 de Ultralytics es sumamente robusta y ha sido diseñada para automatizar la mayoría de la configuración de hiperparámetros.

En algunos casos, como con el *momentum*, a pesar de haber sido configurado un valor, YOLO ignoraba la configuración inicial por encontrar una alternativa mejor, dada la naturaleza de la tarea y el conjunto de datos.

En otros casos, como sucedió con la tasa de aprendizaje (*learning rate*), la configuración manual no ofrecía mejores resultados. El *framework* de forma automática ajusta algunos hiperparámetros que “considera” más adecuados para completar el aprendizaje de forma óptima en el tiempo o épocas de entrenamiento para los que ha sido configurado.

Por otra parte, al tratarse de un problema uniclase, la aplicación de NMS Agnóstico (*agnostic non-maximum suppression*) no representa cambio alguno y se considera irrelevante.

4.4. Desempeño de los mejores modelos

Se entrenaron numerosos modelos con distintos conjuntos de datos y configuración de hiperparámetros. Como resultado de este proceso se desarrollaron los siguientes modelos para las variantes del dataset utilizadas: imágenes a color (RGB) y falso color (ExG+2-PCA+Burn). Los resultados del entrenamiento están disponibles en la tabla 4.3 y la figura 4.2.

TABLA 4.3. Resultados del entrenamiento de modelos.

Métrica	RGB ₈₆	FC ₆₈	Relación
Data Augmentation	9x	3x	+300 %
Tiempo de entrenamiento (h)	0,5	3,5	+14,29 %
Mejor época	17/18	61/359	3,6x
Box Loss	1,224	1,999	-38,79 %
Class Loss	2,168	1,170	+85,25 %
DFL Loss	1,296	1,442	-10,13 %
True Positives	43,89 %	40,00 %	+9,73 %
False Positives	23,02 %	27,36 %	-15,86 %
False Negatives	33,08 %	32,64 %	+1,35 %
mAP@0.5	0,554	0,474	+16,88 %
Precision	0,656	0,594	+10,44 %
Recall	0,570	0,551	+3,45 %
F1-score	0,610	0,571	+6,83 %
F _{1/2} -score	0,637	0,585	+8,89 %
G-measure	0,612	0,572	+6,99 %

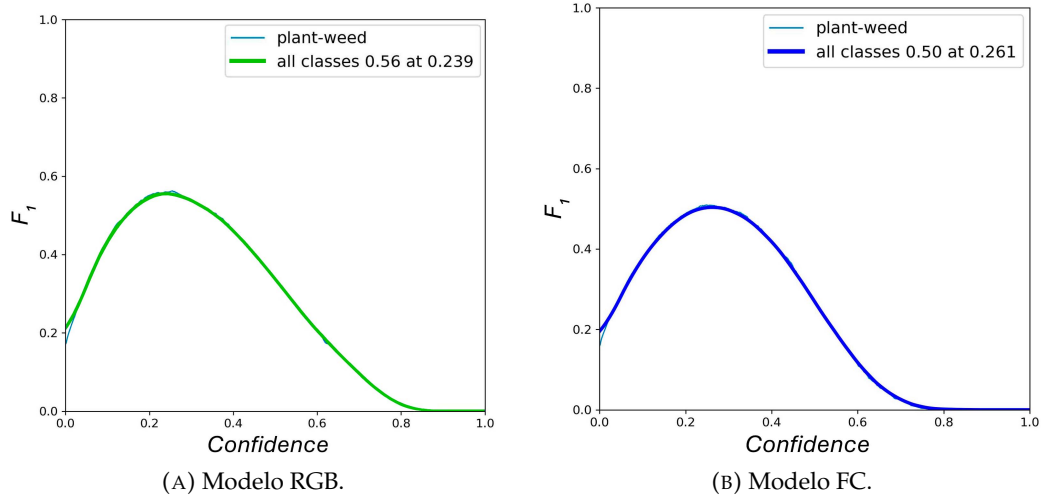


FIGURA 4.2. Curvas de F1-score en función de la confianza.

Para el dataset a color, se detectó que los mejores resultados se alcanzaban al realizar un entrenamiento corto (en torno a 30 minutos), puesto que se trata de un modelo preentrenado. Para mayores tiempos de entrenamiento el modelo se sobrajustaba para los datos de entrenamiento (*overfitting*).

Por el contrario, para el de falso color se hicieron entrenamientos significativamente más largos, dado que era necesario que el modelo aprendiera desde cero.

4.5. Validación de modelos

Se evaluaron los modelos seleccionados con el conjunto de datos de validación y se verificó la efectividad del modelo y las mejoras aportadas por la selección de hiperparámetros, fruto de la gran cantidad de experimentos realizados.

Como puede verse en la tabla 4.4, el modelo con mejor *performance* fue el de imágenes a color RGB, que logró alcanzar un 75 % para *Precision*, $F_{1/2}$ -score de 0,693, 0,574 mAP y *Accuracy* del 44,9 %. El modelo de falso color también ofrece buenos resultados, con 70 % de *Precision*, 40,2 % de *Accuracy*, $F_{1/2}$ -score de 0,643 y mAP de 0,515.

TABLA 4.4. Resultados de la validación de modelos.

Métrica	RGB ₈₆	FC ₆₈	Relación
Tiempo de inferencia (<i>ms</i>)	11,395	12,528	-9,00 %
<i>Accuracy</i>	44,90 %	40,20 %	+11,70 %
<i>Precision</i>	75 %	70 %	+7,10 %
mAP	0,574	0,515	+11,50 %
$F_{1/2}$ -score	0,693	0,643	+7,80 %
<i>G-measure</i>	0,630	0,583	+8,10 %

Si bien en un principio se trabajó con parámetros por defecto para confianza y IoU ($\text{conf}=0,25$ y $\text{iou}=0,6$), luego se determinó que se podían alcanzar mejores resultados al realizar un ajuste fino de estos hiperparámetros: para el modelo RGB, $\text{conf}=0,262$ y $\text{iou}=0,327$; y para el modelo de falso color $\text{conf}=0,290$ y $\text{iou}=0,489$.

De este modo, el modelo es capaz de ofrecer un mejor conteo al no enfocarse en la correcta localización de cada *bounding box*, para centrarse en la efectiva detección de las plantas. Puede observarse la matriz de confusión de cada versión en la figura 4.3.

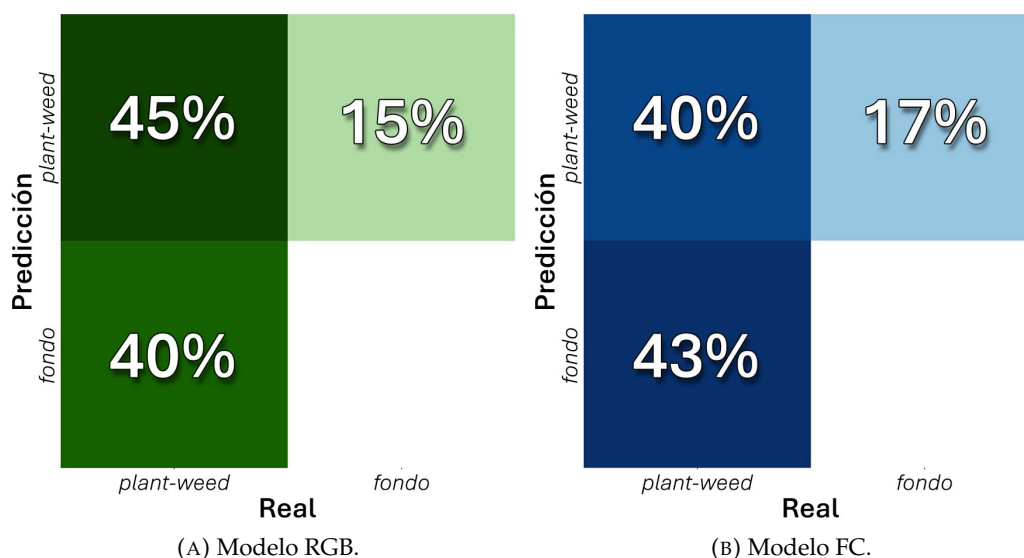


FIGURA 4.3. Matriz de confusión para el conjunto de validación.

4.5.1. Impacto de IoU y confianza

Como se explicó con anterioridad, por la naturaleza misma del desafío que se enfrentaba, era prioritario tener una cifra fiable de plantas detectadas sin sesgar la medición al alza. En la figura 4.4 se observa la evolución de la precisión para cada nivel de confianza del modelo.

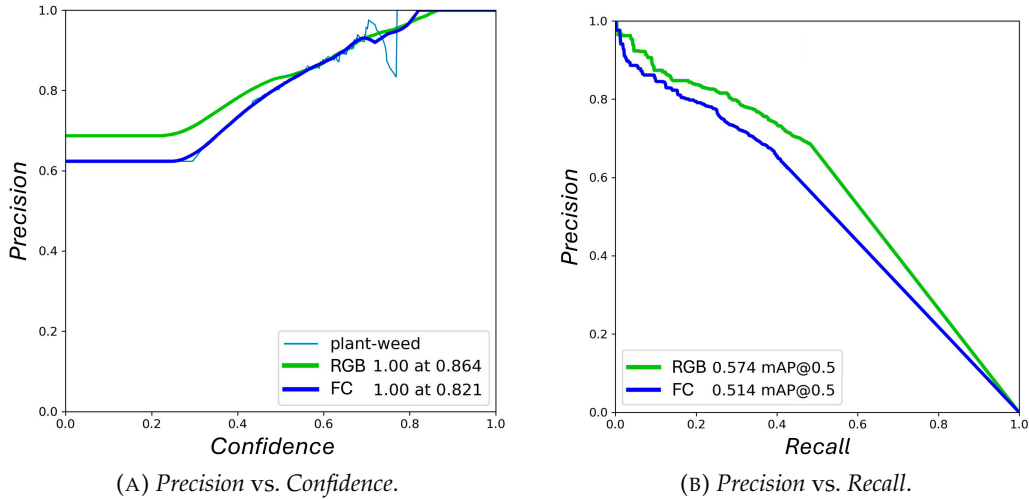


FIGURA 4.4. Curvas de *Precision* en función de la confianza y *recall*.

Al aumentar levemente el umbral de confianza (`conf`), seguridad exigida al modelo para considerar la detección como válida, se logró disminuir significativamente la cantidad de falsos positivos del 21,03 % al 14,79 % (reducción del -29,67 %), para el modelo RGB; y de 27,16 % a 17,20 % (-36,67 %), para el modelo de falso color (ExG+2-PCA+Burn).

De forma complementaria, al reducir el porcentaje de intersección sobre unión, se permitió que el modelo estableciera un equilibrio óptimo entre la sensibilidad para la detección (*recall*) y la precisión de estas (*accuracy*), algo que es crucial para reducir los falsos positivos y lograr un conteo más cercano al número real de objetos.

Con estos ajustes, se sacrificó levemente el mAP (-0,52 %) y algo más el *Accuracy* (-3,65 %), para conseguir una mejora considerable en la *Precision* (+8,85 %) y el *F1/2-score* (+3,90 %). Para el modelo de falso color se consiguió aumentar todas las métricas representativas (con un asombroso +17,45 % para *Precision*) sin sacrificar *Accuracy*.

4.6. Testeo de modelos

Por último, se realizó la comparación del desempeño del modelo contra el conjunto de testeo. Puede verificarse en la tabla 4.5 que los resultados fueron semejantes a los logrados para el conjunto de validación.

En la figura 4.5 se aprecia que la elección del *F1/2-score* como métrica de diseño permitió desarrollar modelos que no solo generalizan los resultados, sino que lo hacen sin aumentar la cantidad de falsos positivos (FP).

TABLA 4.5. Resultados del testeo de modelos.

Métrica	RGB ₈₆	FC ₆₈	Relación
<i>Accuracy</i>	42,1 %	37,6 %	+12 %
<i>Precision</i>	77,8 %	75,3 %	+3,7 %
mAP	0,563	0,476	+18,3 %
$F_{1/2}$ -score	0,691	0,654	+5,7 %
<i>G-measure</i>	0,610	0,568	+7,4 %

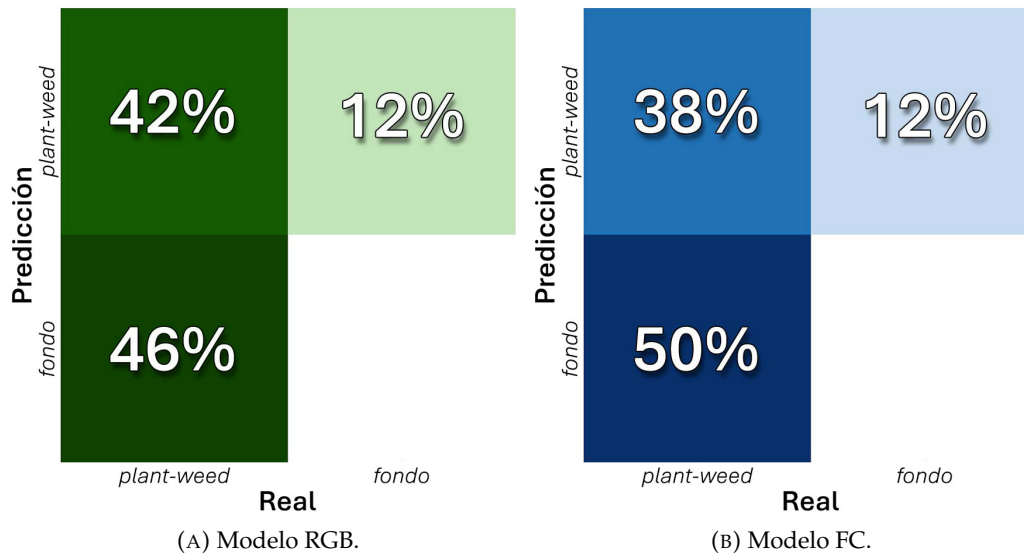


FIGURA 4.5. Matriz de confusión para el conjunto de testeo.

4.6.1. Impacto del desarrollo de la planta en la detección

Se realizó además el testeo de los modelos evaluados exclusivamente con subconjuntos de imágenes de igual estadio de desarrollo de la planta (temprano y avanzado) para observar el rendimiento en cada escenario (figura 4.6).

Se puede apreciar que el modelo RGB ofreció resultados significativamente mejores para la planta en desarrollo temprano. En la tabla 4.6 se ve que para estadios más avanzados, se deteriora progresivamente la capacidad de detección del modelo y sufre una fuerte caída del *Accuracy* producida por un notable incremento de los falsos negativos. Se logró mantener constantes los falsos positivos, evitando generar un sesgo al alza en la medición de la densidad.

TABLA 4.6. Respuesta del modelo RGB en función del desarrollo y con terreno anegado.

Métrica	RGB ₈₆ (Referencia)	Desarrollo temprano	Desarrollo avanzado	Lote anegado
<i>Accuracy</i>	42,1 %	47,4 %	33,2 %	48,9 %
<i>Precision</i>	77,8 %	70,9 %	63,6 %	86,6 %
Recall	47,8 %	58,8 %	41,0 %	52,9 %
mAP	0,563	0,563	0,445	0,660
$F_{1/2}$ -score	0,691	0,681	0,573	0,768
<i>G-measure</i>	0,610	0,646	0,510	0,677

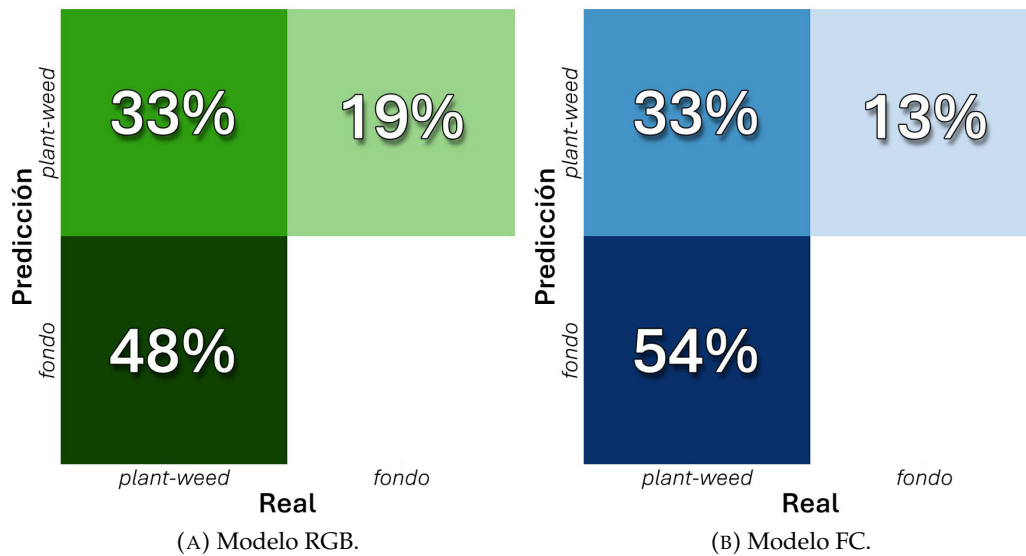


FIGURA 4.6. Matriz de confusión para planta con desarrollo avanzado.

Sin embargo, puede verse que al combinar ambos subconjuntos (referencia), el modelo logró un comportamiento más equilibrado en las detecciones, que tuvo un impacto positivo en precisión y $F_{1/2}$ -score, sin perjudicar al mAP y promediando el *Accuracy*.

Por otro lado, se observó un comportamiento similar para el dataset de falso color (FC), pero con la particularidad de desempeñarse mucho mejor para plantas con desarrollo avanzado, al lograr reducir los falsos positivos a la mitad sin sacrificar *Accuracy*. Se puede ver en la tabla 4.7 que logró alcanzar un 72,6 % de *Precision*, lo que representa +12,7 % de mejora respecto al modelo RGB y +7,5 % para $F_{1/2}$ -score.

Esto evidencia que el modelo de falso color aquí propuesto representa un buen *trade-off* entre tiempo de cómputo y precisión de la medición para plantas en etapas de desarrollo avanzado.

TABLA 4.7. Respuesta del modelo FC en función del desarrollo.

Métrica	FC ₆₈ (Referencia)	Desarrollo temprano	Desarrollo avanzado
<i>Accuracy</i>	37,6 %	42,7 %	33,4 %
<i>Precision</i>	75,3 %	71,7 %	72,6 %
Recall	42,9 %	51,3 %	38,2 %
mAP	0,528	0,539	0,487
$F_{1/2}$ -score	0,654	0,664	0,616
<i>G-measure</i>	0,568	0,606	0,527

Por último, se realizó una prueba de testeo adicional con el modelo RGB para imágenes de un lote anegado. El modelo no fue entrenado con datos con esas características, por lo que representa un buen parámetro de la variabilidad a la que podría enfrentarse el modelo en situaciones productivas. Como puede verse en la tabla 4.6, la *performance* fue incluso mejor que para cultivos en condiciones normales. El modelo ofreció un 86,6 % de precisión (+22 %), 48,9 % de *Accuracy*, con 0,66 de mAP (+17 %) y $F_{1/2}$ -score 0,768. Esto probablemente se deba a que,

al estar el terreno inundado, es más fácil diferenciar la planta del fondo que en condiciones normales. Puede verse un ejemplo ilustrativo en la figura 4.7.



FIGURA 4.7. Ejemeplo de inferencia para lote con anegamiento.

4.7. Desempeño en producción

Se seleccionaron 35 imágenes adicionales para emular el desempeño del modelo en condiciones productivas. Todos estos eran datos que no habían sido vistos con anterioridad por el modelo, ya que no formaron parte de los conjuntos de entrenamiento ni validación.

Se realizaron pruebas para distintas etapas de desarrollo de la planta, alturas de vuelo y diversas situaciones problemáticas (anegamiento, presencia de huellas e imágenes desenfocadas). Se evaluó también el impacto de la configuración de IoU y *Confidence* en las predicciones realizadas por el modelo.

Estas pruebas se realizaron con el *pipeline* de inferencia (figura 3.1) y se envió cada imagen para que se realice el conteo de plantas. La medición fue contrastada con los datos tomados a campo (conteo de aro) y la medición de densidad del lote provista por el cliente (*ground truth*). Para medir la diferencia entre cada medición se calculó la raíz del error cuadrático medio (RMSE).

Al realizar una inspección visual general de las detecciones, se verificó la coherencia de estas dada la distribución del cultivo y la correcta discriminación de malezas. Todas corresponden efectivamente a plantas (no se observaron detecciones de rastrojo u otros objetos extraños como falsos positivos).

Es importante destacar que la medida de error es meramente orientativa, ya que:

1. Para poder medir realmente la efectividad en condiciones productivas del método propuesto, sería necesario realizar experimentos diseñados para tal

fin: tomar todas las muestras necesarias de un mismo lote y realizar el procesamiento en *batch*.

2. El conteo del aro, si bien es preciso, no es representativo de la densidad real del lote, por tratarse de una única muestra con una superficie extremadamente reducida.
3. La medición de referencia provista por el cliente posee un gran margen de error y no se tiene certeza de la representatividad estadística de dicha medición (no se ha informado el número total de muestras tomado durante la medición tradicional).

Al momento de redactar la memoria, no se contaba con datos que cumplieren con estos requisitos, por lo que no fue posible realizar pruebas representativas. Fue por esto que se optó simplemente por verificar la respuesta del sistema ante condiciones diversas, sin evaluar la precisión de la medición estadística *per se*.

4.7.1. Caracterización del sistema productivo

La distinción tradicional entre sistemas agrícolas extensivos e intensivos se vuelve difusa ante modelos de producción a gran escala que incorporan un alto nivel de tecnología y manejo intensivo. Los sistemas extensivos tradicionalmente se caracterizan por abarcar grandes superficies con bajos insumos por hectárea y menor tecnificación, con el objetivo de maximizar la producción total más que el rendimiento unitario.

Sin embargo, el modelo productivo del cliente contrasta con esta categorización. A pesar de operar sobre una gran superficie, propia de un sistema extensivo, su manejo agronómico es fundamentalmente intensivo. Este carácter se manifiesta en el uso avanzado de tecnología y la búsqueda constante de alta productividad. El objetivo principal es maximizar la eficiencia y el rendimiento por hectárea.

Por lo tanto, la combinación de la vasta escala con prácticas agrícolas de alta intensidad tecnológica y de insumos obliga a considerar estos modelos como predominantemente intensivos, a pesar de su dimensión espacial.

Los *papers* consultados buscaron resolver el conteo de plantas en parcelas de ensayo con menor superficie o en cultivos con densidades y cantidades de plantas significativamente menores. Es fundamental destacar que la herramienta aquí desarrollada, al ser una herramienta productiva que busca cubrir grandes extensiones de cultivo (lotes de 200 ha) con gran densidad de plantas (200 plantas/ha), debió afrontar desafíos significativamente más complejos al que representan las parcelas de ensayo. Esto constituye un punto diferencial de la solución provista en este trabajo respecto al estado del arte.

Crítica del método tradicional

El método de conteo tradicional implica recorrer el lote a pie o en moto y tomar al menos 10 muestras con un aro de 0,25 m². Según lo informado por el cliente, el conteo del aro ofrece una precisión estimada de ± 10 plantas sobre la densidad ideal de 200 plantas/m². Esto se traduce en un margen de error del $\pm 5\%$. Como se detalla en el apéndice B, para alcanzar un nivel de confianza del 90 %, se necesitarían un mínimo de 17 aros, mientras que para un 95 % de confianza, se

requeriría alcanzar los 24 aros. Según lo informado, por la limitación de recursos y la cantidad de hectáreas a cubrir, actualmente realizan la medición con 10 muestras, lo que ofrece un nivel de confianza del 80 %.

Según lo calculado en el apéndice B, para realizar una medición equivalente con el modelo propuesto, dada la precisión alcanzada, sería recomendable capturar al menos 55 muestras de un mismo lote para lograr una medición con un margen de error del $\pm 5\%$ y un 80 % de confianza.

4.7.2. Impacto del desarrollo de la planta en la *performance*

Se verifica que los mejores resultados se obtienen para las plantas en etapa de desarrollo temprano (209) y avanzado (503), donde las detecciones tienen una relación unívoca con las plantas presentes en la imagen.

Se verificó que para etapas tardías de desarrollo (mayor a 3 semanas) los resultados ofrecidos por el modelo no son de utilidad, ya que se vuelve incapaz de reconocer plantas individuales (debido al macollaje). Únicamente reconoce conglomerados de plantas (*clusters*), lo que imposibilita la medición de densidad.

4.7.3. Reflexión sobre el umbral de confianza y IoU

Es posible realizar una comparación entre la medición “optimizada” (con umbrales de IoU y *confidence* personalizados) y las detecciones por defecto del modelo. Al realizar una inspección visual de la figura 4.8 y comparar las detecciones para cada caso, se puede observar que al ajustar los umbrales (puntos rojos), se consigue filtrar mejor los falsos positivos al evitar contar varias veces la misma planta. En la detección estándar (puntos azules) se puede ver una gran aglomeración de detecciones en regiones de la imagen donde hay pocas plantas.

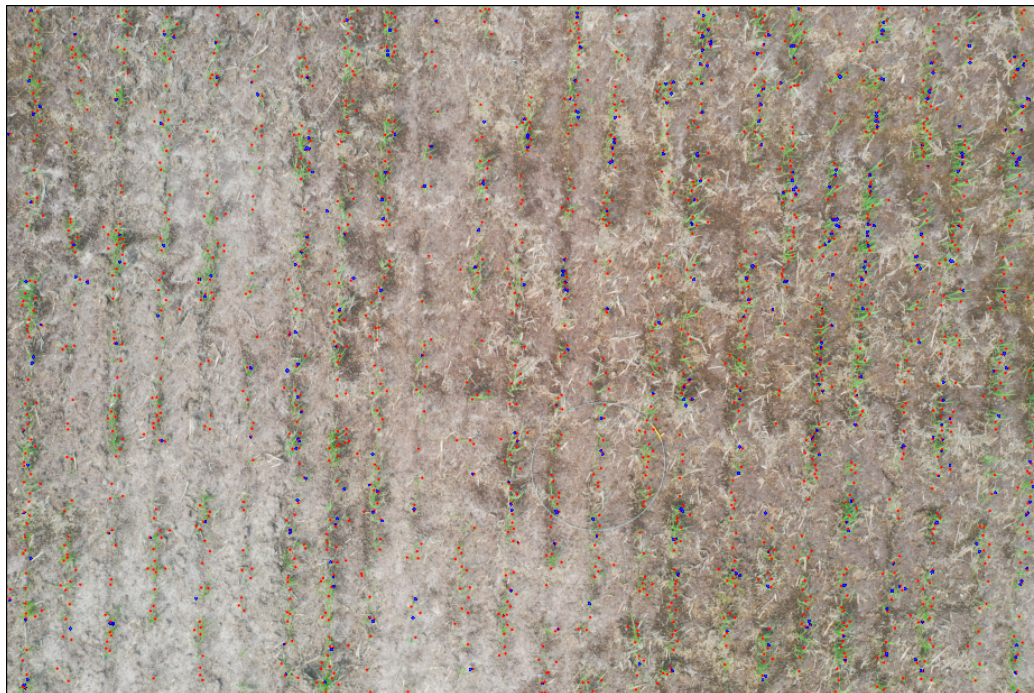


FIGURA 4.8. Impacto del IoU y *Confidence* en la inferencia.

4.7.4. Casos problemáticos

Las inferencias realizadas para situaciones problemáticas comunes ofrecieron resultados equivalentes a los alcanzados en situaciones normales, lo que demuestra que las estrategias de diseño implementadas para tal fin fueron efectivas.

En particular, el modelo fue capaz de realizar el conteo en lotes con anegamiento, rastrojo o huellas de maquinaria y operarios, sin ver afectada la medición, como se ve en la figura 4.7.

Además, respondió adecuadamente a imágenes con un elevado nivel de desenfoque, generalmente producido por la presencia de vientos fuertes durante la captura, que impiden al dron estabilizarse lo suficiente como para lograr un enfoque óptimo.

4.7.5. Impacto de la altura de vuelo en la *performance*

Por último, se realizaron conteos para un mismo punto del lote fotografiados a distintas alturas a intervalos de 0,5 metros para evaluar el impacto de la altura de vuelo en la *performance*. Pueden verse los resultados detallados en la tabla A.2.

Si bien el modelo fue entrenado con imágenes capturadas a una altitud de 3,5 metros, es posible realizar la inferencia en un rango de alturas sin que esto genere un impacto negativo en la medición.

Se verificó un rango útil para la inferencia entre alturas de 1,5 a 6,5 m, con valores óptimos en torno a 2 y 4,5 m para lograr un buen balance entre detalle y superficie muestreada. A partir de los 7 m de altura se observa un claro deterioro del número de detecciones.

El impacto de la altura es más notorio para plantas en etapa temprana de desarrollo (por ser más pequeñas). Para plantas con un desarrollo avanzado, es posible incrementar levemente la altura de vuelo y acercarse a los límites del rango útil antes mencionado sin perder precisión.

4.8. Estimación de recursos necesarios

Según el cálculo realizado en el apéndice B, sería necesario tomar al menos 30 muestras (imágenes de un mismo lote) para realizar una medición estadísticamente representativa de un lote estándar de 200 ha.

Dada la precisión actual alcanzada por el mejor modelo, se recomienda trabajar con un número de 55 imágenes para lograr una medición con niveles de confianza y margen de error equivalentes a los obtenidos por el cliente con métodos tradicionales.

Si se miden los tiempos de proceso para cada una de las etapas, se puede realizar una estimación de los recursos necesarios para procesar un *batch* de n muestras de un mismo lote.

Como se puede ver en la tabla 4.8, capturar 30 imágenes requeriría ~ 22 min para el vuelo del dron y ~ 9 min de procesamiento de las imágenes para medir la densidad de plantas, es decir, un poco más de media hora para completar la tarea. Esto representa una disminución del tiempo necesario para realizar la tarea de hasta el 87,5 % respecto a los métodos tradicionales.

La autonomía de vuelo para el DJI Mavic 2 Pro es de 30 min, pero es recomendable no extender el vuelo más allá de los 25 min para que el dron pueda volver a la base y realizar un aterrizaje seguro. Dentro de esta ventana temporal, es posible incrementar el número de muestras sin necesidad de realizar un recambio de batería (demora un estimado de 7 min).

Si bien al incrementar el número de fotos el tiempo de vuelo se extiende (porque el dron necesita estabilizarse antes de tomar la fotografía, lo que disminuye la velocidad media de vuelo), esto no representa un incremento significativo.

De la tabla 4.8 se deduce que si se incrementara el número de muestras de 30 a 55 (+83 %), el tiempo de cómputo aumentaría apenas 6 min (+65 %). En este caso, el tiempo de vuelo se incrementaría en 18 min porque, al superarse el umbral de 25 min, es necesario realizar un cambio de batería.

Gracias a este comportamiento no lineal, el método es fácilmente escalable, lo que permite lograr un buen equilibrio entre el tiempo dedicado a la tarea y la precisión deseada en la medición.

Por otro lado, si se desea utilizar el modelo alternativo para imágenes de falso color, el tiempo de cómputo se duplica por las múltiples etapas de transformación de imágenes necesarias. Como se ve en la tabla 4.8, el tiempo total para 30 imágenes ascendería en este caso a 41 min (resultado de 31, 2 + 10, 4 min).

Además, es recomendable realizar la inferencia de forma local, ya que el procesamiento a través de la nube fue lo que más incrementó el tiempo de procesamiento (~6x). Al realizar el cómputo en la nube (+45 min) o requerir la transferencia de los archivos vía Internet (+6 min), la duración de la tarea de conteo puede ascender a más de 1 hora para 30 imágenes o hasta 2 ½ horas para 55 fotografías, lo que representa un incremento de 6x. Puede verse en detalle la duración de cada proceso en la tabla A.1 del apéndice.

En cuanto al uso de almacenamiento, se necesita disponer de al menos 150 MB para almacenar 30 imágenes (de 5 MB cada una) y un espacio temporal adicional de +11,4 MB/imagen durante el procesamiento. Es decir, un espacio total en disco de 492 MB. Para 50 muestras, serían necesarios 250 MB para las imágenes y 820 MB para los archivos temporales. Ambos son valores razonables para las capacidades de los equipos estándar actuales.

TABLA 4.8. Estimación del tiempo para la tarea de conteo.

Proceso	Lote 30 imágenes	Lote 55 imágenes	Incremento
Vuelo de dron	22 min	40 min	1,82x
Transferencia ($\downarrow 300$ $\uparrow 50$ Mbps) *	4,5 $\pm$ 1,5 min	8,3 $\pm$ 2,8 min	1,83x
Pipeline de inferencia	9,2 min	15,2 min	1,65x
Procesamiento en local (RGB)	31,2 min	55,2 min	1,77x
Adicional ExG+2-PCA	+10,4 min	+19 min	1,83x
Adicional cloud * (cómputo + transferencia)	+49,5 min	+89 min	1,80x

4.8.1. Aporte a la productividad

Se puede concluir entonces que es factible incrementar el número de imágenes de un mismo lote según el margen de error deseado para la medición y los recursos disponibles para realizar la tarea.

Es razonable asumir un tiempo total menor a 1 hora para finalizar la tarea de conteo (vuelo de drones + cómputo). Esto representa una mejora de la eficiencia de al menos 5x respecto al conteo con métodos tradicionales.

La solución propuesta para el conteo de plantas en cultivos de arroz mediante la utilización de un *pipeline* de ML aporta las siguientes soluciones:

- Reduce la necesidad de contar con personal capacitado, ya que se puede programar con antelación el vuelo del dron.
- Elimina el factor humano, que pudiera haber introducido un sesgo en la medición.
- Eficientiza la tarea a campo, al limitarla al vuelo del dron, sin necesidad de ingresar al lote.
- Permite ser fácilmente escalada con el aumento de las hectáreas producidas.

Capítulo 5

Conclusiones

Este capítulo detalla las conclusiones extraídas de los resultados del presente trabajo en el problema del conteo de plantas en cultivos de arroz.

Los hallazgos validan la viabilidad del método propuesto y logran un balance efectivo entre precisión en la medición de densidad y optimización de recursos.

5.1. Conclusiones generales

A pesar de la complejidad del problema afrontado, los resultados obtenidos son prometedores. La solución desarrollada en este trabajo permite facilitar el conteo de plantas en lotes de gran extensión y alcanzar un buen balance entre la precisión de la medición de densidad de plantas y el uso de recursos humanos y materiales.

El modelo responde satisfactoriamente en las condiciones para las que fue entrenado (plantas en etapa de desarrollo temprano o intermedio). Además, es robusto a la variabilidad de las condiciones de entrada. Ofrece una *performance* adecuada en condiciones de iluminación variable, con lotes anegados o ante la presencia de objetos extraños (como piedras, caracoles o rastrojo) y huellas de operarios y maquinaria. Incluso es capaz de realizar la detección de plantas con imágenes desenfocadas.

La aplicación en conjunto de los modelos propuestos aporta una solución versátil, robusta y eficiente para distintos estadios de desarrollo y cultivos con características variables. La solución propuesta trabaja bajo requerimientos de tiempo y de hardware prácticos, que permiten incrementar la productividad en más de un 500 %.

Si bien existe aún margen de mejora del modelo, principalmente si se nutre el dataset con un mayor número y diversidad de datos, se ha logrado probar la viabilidad del método propuesto y se ha abierto el camino a futuras líneas de investigación para soluciones superadoras.

Además, dentro del marco de este trabajo se desarrollaron técnicas innovadoras para el procesamiento de imágenes de cultivos a través de la combinación de índices de vegetación (ExG), reducción dimensional (PCA) y técnicas de procesamiento de imágenes (combinación, fusión, CLAHE y máscaras). Fruto de esta exploración se creó un conjunto de datos derivados que permite diferenciar con más facilidad las plantas del fondo.

5.2. Próximos pasos

A futuro, sería interesante poder trabajar tan solo con 30 muestras para poder realizar toda la captura de imágenes durante un único vuelo de dron (sin necesidad de cambiar la batería). De este modo, se podría reducir el tiempo total de la tarea a menos de 30 minutos.

Si bien el modelo alternativo propuesto que trabaja con imágenes de falso color no logró superar al de imágenes RGB, los resultados obtenidos con este procesamiento son prometedores y permiten suponer que, si continúa el desarrollo de este modelo, se podrían llegar a mejorar ampliamente los resultados actuales.

Además, se pudo observar que la aplicación de PCA permitió separar de forma óptima el fondo (primer componente) de las plantas (segundo componente). Es de suponer que se pueda aplicar esta técnica de formas diferentes a las exploradas durante este trabajo.

Algunas rutas de exploración para futuros trabajos incluyen:

- Continuar el análisis del dataset de falso color (PCA + ExG + *Burn Blend*) para llegar a mejores resultados.
- Enriquecer el dataset con nuevas imágenes que reúnan las características ideales definidas en el presente trabajo.
- Incorporar al conjunto de entrenamiento un mayor número de ejemplos de situaciones problemáticas (como pueden ser rastrojo, anegamiento, lotes con gran presencia de maleza), distintas situaciones de ambientación (características del cultivo) o más imágenes de suelo sin plantas.
- Para mejorar la calidad del etiquetado se propone:
 - Trabajar exclusivamente en etapa temprana de desarrollo de la planta.
 - Revisar los casos críticos con un especialista.
 - Evaluar la utilización de imágenes de falso color durante el etiquetado.
- Investigar avances en el estado del arte e implementar otras arquitecturas.
- Explorar la incorporación de modelos de segmentación semántica o arquitecturas híbridas que combinen detección y segmentación.
- Experimentar con modelos YOLOv8 de mayor escala (*Large* o *Extra Large*).
- Evaluar la utilización de etiquetas `cluster` de plantas (plantas superpuestas) para desarrollar un modelo en paralelo capaz de detectar puntos potencialmente problemáticos para el conteo.
- Analizar la incorporación de algún tipo de heurística para etiquetas de baja confianza (`occluded`) o implementar técnicas de procesamiento especiales para estos objetos problemáticos.
- Realizar pruebas productivas a campo con nuevas imágenes de drones para evaluar el rendimiento del modelo en entornos productivos.
- Capturar las múltiples muestras necesarias para uno o varios lotes, y así comprobar de forma experimental el desempeño del *pipeline* de inferencia durante el procesamiento en lotes.

Apéndice A

Resultados de inferencia

En la tabla A.1 se presentan en detalle las etapas del *pipeline* de inferencia y la duración de cada tarea durante el cómputo de una muestra individual, y con lotes de 30 y 55 imágenes.

TABLA A.1. Tiempo de cómputo para procesamiento en lote.

Etapa	Tarea	Tiempos por imagen	Lote 30 imágenes	Lote 55 imágenes
Pre-proceso	ExG+2-PCA **	20,8 seg	624 seg	1144 seg
	<i>Tiles splitter</i>	500 ms	15 seg	27,5 seg
	CLAHE	486 ms	14,58 seg	26,73 seg
	<i>Upload *</i>	40 seg	1200 seg	2200 seg
Modelo YOLOv8	Inicialización	60 seg/lote	60 seg	60 seg
	<i>Extra cloud *</i>	70 seg/lote	70 seg	70 seg
	Inferencia	2341,6 ms	60 seg	110 seg
	<i>Download *</i>	47 seg	1410 seg	2585 seg
Pos-proceso	<i>Tiles un-splitter</i>	4 seg	120 seg	220 seg
	<i>Line detection</i>	8,5 seg	255 seg	467,5 seg
Procesamiento en local		15,8 seg	9,2 min	15,2 min
Adicional <i>cloud *</i> (cómputo + transferencia)			+45 min (5,9x)	+81 min (6,3x)
Adicional ExG+2-PCA **			+10,4 min (2,1x)	+19 min (2,3x)

En la tabla A.2 se encuentran los resultados entregados por el modelo para imágenes capturadas en un mismo sector del lote a diferentes alturas (una medición con una única muestra del lote). Se puede observar que el RMSE entre la densidad calculada por el modelo y la medición —para el lote completo— informada por el cliente (*ground truth*, *GT*), se obtienen valores de error dependientes de la altura de vuelo con la que se tomaron las imágenes.

Si se toma como punto de comparación la densidad medida con el aro de medición (cobertura de $\frac{1}{4}$ m²) para el mismo sector del lote y se la contrasta con el GT para el lote completo, se obtiene un RMSE de 36. Este valor es razonable dada la variabilidad de densidad de los cultivos dentro de un mismo lote y se compensa de forma estadística al repetir la medición para múltiples aros equidistribuidos.

En este ejemplo puntual, el valor mínimo de error se obtuvo para una altura de 2,5 m. De forma experimental, se determinó un rango útil entre 2 y 4 metros para obtener un buen balance entre el área cubierta y la resolución por píxel.

Si se compara el RMSE del aro con el de la medición del modelo, se puede apreciar que en este ejemplo, para alturas cercanas a 2,5 m, se obtuvieron errores menores a los de la medición en el aro.

Es importante destacar que esta prueba fue realizada para una única muestra del lote por falta de disponibilidad de datos. Como se explica en las conclusiones del trabajo, para realizar estas mediciones de forma precisa, sería necesario capturar en un entorno productivo el número de muestras requeridas por el modelo y, además, realizar un análisis exhaustivo del margen de error del método tradicional.

TABLA A.2. Detecciones para un mismo sector a distintas alturas.

Altura de vuelo (metros)	Área cubierta (m ² /img)	Detecciones (plantas/img)	Densidad calculada (plantas/m ²)	Error RMSE (Modelo vs. GT)	Rango útil de medición
1	1,0	392	385,6	205,6	✗
1 ½	2,3	636	245,3	65,3	✓
2	4,4	941	213,3	33,3	✓
2 ½	6,8	1129	167,2	12,8	✓
3	9,9	1324	133,4	46,6	✓
3 ½	13,0	1439	111,0	69,0	✓
4	16,8	1523	90,7	89,3	✗
4 ½	21,0	1612	76,7	103,3	✗
5	25,4	1617	63,6	116,4	✗
5 ½	35,2	1533	43,6	136,4	✗
6	41,8	1571	37,6	142,4	✗
6 ½	46,4	1514	32,6	147,4	✗
7	55,0	1395	25,4	154,6	✗
7 ½	64,0	1276	19,9	160,1	✗
8	70,6	1145	16,2	163,8	✗
8 ½	78,2	1043	13,3	166,7	✗
9	90,5	887	9,8	170,2	✗
9 ½	101,7	817	8,0	172,0	✗
10	110,3	784	7,1	172,9	✗

Apéndice B

Número de muestras y precisión en la medición

Desarrollo estadístico

Parámetros informados por el cliente

- Número de muestras: 10 aros
- Área de muestreo: $0,25 \text{ m}^2$
- Área del lote: $\sim 200 \text{ ha} = 200 \cdot 10\,000 \text{ m}^2 = 2\,000\,000 \text{ m}^2$
- Variabilidad: la densidad varía entre 150 a 250 plantas/ m^2 para distintos sectores del lote.
- Precisión (Margen de Error): $\pm 10/200 = \pm 5 \%$
- Nivel de Confianza: se considerarán niveles del 90 % y 95 %.

B.1. Muestreo por conglomerados (*Cluster sampling*)

En el muestreo por conglomerados en una etapa, donde se muestrean todos los elementos dentro de los conglomerados seleccionados, la población se divide en M conglomerados. La fórmula para el número de conglomerados (m) a seleccionar es diferente, y la precisión depende de la variabilidad entre conglomerados.

La fórmula del número de muestras (m) a tomar para estimar la media poblacional con un error absoluto E es:

$$m = \frac{Z^2 \cdot S_c^2}{E^2 + \frac{Z^2}{M} \cdot S_c^2}$$

Donde:

- M es el número total de *clusters* en la población (muestras tomadas),
- S_c^2 es la varianza estimada de los *clusters* promedio,
- E es el margen de error absoluto deseado,
- Z es el ponderador para el nivel de confianza deseado.

Unidad de muestreo (*cluster*) y número de conglomerados

Dado que cada muestra se toma con un aro de $0,25 \text{ m}^2$, el número de conglomerados (*clusters*) será:

$$M = \frac{\text{Área total}}{\text{Área del cluster}} = \frac{2\,000\,000 \text{ m}^2}{0,25 \text{ m}^2} = 8\,000\,000 \text{ clusters}$$

Varianza estimada

Basándose en el rango observado (150 a 250 plantas/ m^2) en la densidad por m^2 en diferentes sectores del lote,

$$\Delta x = x_{\max} - x_{\min} = 250 - 150 = 100 \text{ plantas/m}^2$$

y asumiendo que representa aproximadamente 4 desviaciones estándar, la desviación de los *clusters* ($S_{\bar{c}}$) será:

$$S_{\bar{c}} \approx \frac{\Delta x}{4} = 25 \text{ plantas/m}^2$$

Número de muestras

Sabiendo que el error absoluto $E = 10 \text{ plantas/m}^2$.

Caso 1

Para tener un 90 % de Confianza, $Z = 1,645$:

$$m = \frac{Z^2 \cdot S_{\bar{c}}^2}{E^2 + \frac{Z^2}{M} \cdot S_{\bar{c}}^2}$$

$$m_{90} = \frac{(1,645)^2 \cdot (25)^2}{10^2 + \frac{(1,645)^2}{8 \cdot 10^6} \cdot (25)^2} = \frac{2,706025 \cdot 625}{100 + \frac{2,706025}{8 \cdot 10^6} \cdot 625} = \frac{1691,265625}{100,000211408} \approx 16,91$$

Redondeando al entero más próximo, el número de muestras necesarias para obtener una medición con un 90 % de confianza es de 17 aros.

Caso 2

Para un nivel de confianza del 95 %, $Z = 1,96$:

$$m_{95} = \frac{(1,96)^2 \cdot (25)^2}{10^2 + \frac{(1,96)^2}{8 \cdot 10^6} \cdot (25)^2} = \frac{3,8416 \cdot 625}{100 + \frac{3,8416}{8 \cdot 10^6} \cdot 625} = \frac{2401}{100,00030125} \approx 24,0099$$

Es decir, para una medición con el 95 % de confianza serán necesarios 24 aros.

Caso 3

Tomando la cantidad de aros informada por el cliente, podemos calcular también el nivel de confianza de su medición.

Despejando Z :

$$\begin{aligned}
 m &= \frac{Z^2 \cdot S_c^2}{E^2 + \frac{Z^2}{M} \cdot S_c^2} \\
 m \left(E^2 + \frac{Z^2}{M} \cdot S_c^2 \right) &= Z^2 \cdot S_c^2 \\
 mE^2 + m \frac{Z^2}{M} \cdot S_c^2 &= Z^2 \cdot S_c^2 \\
 mE^2 &= Z^2 \cdot S_c^2 - m \frac{Z^2}{M} \cdot S_c^2 \\
 mE^2 &= \left(1 - \frac{m}{M} \right) Z^2 \cdot S_c^2 \\
 Z^2 &= \frac{mE^2}{S_c^2 \left(1 - \frac{m}{M} \right)}
 \end{aligned}$$

Substituyendo los valores $E = 10$, $S_c = 25$, $M = 8 \cdot 10^6$:

$$\begin{aligned}
 Z_{m=10}^2 &= \frac{10 \cdot 10^2}{25^2 \left(1 - \frac{10}{8 \cdot 10^6} \right)} = \frac{10^3}{625 \left(1 - \frac{1}{8 \cdot 10^5} \right)} = \frac{1,6}{0,99999875} \approx 1,600002 \\
 \Rightarrow Z_{m=10} &= \sqrt{1,600002} \approx 1,26491185
 \end{aligned}$$

Consultando el valor en una tabla de distribución normal estándar [46], podemos ver que la probabilidad total acumulada para el valor $Z_{m=10}$ corresponde a $\approx 0,89617$. Dado que la distribución normal es simétrica, la probabilidad acumulada desde la media es:

$$0,89617 - 0,5 = 0,39617$$

Entonces, el área para ambas colas de distribución —fuera del intervalo de confianza— es:

$$2 \times (0,5 - 0,39617) = 2 \times 0,1038 = 0,2056$$

En conclusión, el Nivel de Confianza —área dentro del intervalo— para 10 muestras será:

$$1 - 0,2056 = 0,7944 \approx 80 \%$$

Relación entre *Accuracy* y tamaño muestral

La solución propuesta en el trabajo de tesis consiste en la utilización de un modelo de Visión Computacional de detección de objetos que:

1. Identifica la ubicación de las plantas.
2. Cuenta el número total de detecciones .
3. Calcula la densidad de plantas, conociendo el área cubierta por la imagen aérea.

El objetivo del presente anexo es, con las métricas del modelo (*accuracy*, *precision* y *recall*), intentar estimar la cantidad de muestras necesarias (fotos de dron) para realizar una medición con un determinado intervalo de confianza.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN}$$

Donde:

- TP son los Verdaderos Positivos,
- FP son los Falsos Positivos,
- TN son los Verdaderos Negativos,
- FN son los Falsos Negativos.

Relación entre *Accuracy* y proporción estimada

Dentro del contexto del *Machine Learning*, se puede adoptar un enfoque estadístico considerando las n inferencias realizadas por el modelo para un mismo lote como una muestra aleatoria de predicciones, donde cada predicción puede ser correcta o incorrecta.

Esto se alinea con el concepto de ensayos de Bernoulli.

Donde:

$$P(X = x) = p^x(1 - p)^{1-x}$$

- $P(X = x)$ es la probabilidad de que la variable aleatoria X tome el valor x ,
- x es el resultado del intento, que solo puede ser 0 (fracaso) o 1 (éxito),
- p es la probabilidad de éxito en un único intento (valor entre 0 y 1),
- $1 - p$ es la probabilidad de fracaso.

Puede desglosarse en dos casos:

$$P(X = 1) = p \quad P(X = 0) = 1 - p$$

Con función de distribución:

$$F(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 - p & \text{si } 0 \leq x < 1 \\ 1 & \text{si } x \geq 1 \end{cases}$$

Y parámetros:

$$E[X] = p \quad \text{Var}[X] = p(1 - p)$$

Partiendo de la premisa previa de que el cultivo presenta una distribución normal, es posible realizar una aproximación desde la distribución binomial, principio fundamentado en el Teorema de De Moivre-Laplace.

Entonces, cada inferencia para el mismo lote puede modelarse como un ensayo de Bernoulli con una probabilidad de éxito p (acierto).

Como consecuencia, el número total de predicciones correctas seguirá una distribución binomial $B(n, p)$.

Intervalo de Wald (aproximación normal)

Siempre y cuando se cumplan las premisas del teorema de De Moivre-Laplace y n sea suficientemente grande, la distribución binomial puede aproximarse mediante una distribución normal con media $\mu = np$ y varianza $\sigma^2 = np(1 - p)$.

La fórmula del Intervalo de Wald utiliza esta aproximación para construir un intervalo de confianza para la proporción poblacional p basándose en la proporción muestral de predicciones correctas.

Esta puede definirse como $\hat{p} = \frac{\text{aciertos}}{n}$ con distribución aproximadamente normal con media p y error estándar $SE = \sqrt{\frac{p(1-p)}{n}}$.

Para construir el intervalo de confianza $1 - \alpha$, utilizamos la expresión $Z_{\alpha/2}$ tal que $P(-Z_{\alpha/2} \leq Z \leq Z_{\alpha/2}) = 1 - \alpha$, siendo Z la variable normal estándar.

Entonces, el intervalo puede definirse como:

$$P\left(\hat{p} - Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} \leq p \leq \hat{p} + Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}\right) = 1 - \alpha$$

Dentro de estos supuestos se cumple que:

$$\forall n\hat{p} \geq 5 \quad \wedge \quad n(1 - \hat{p}) \geq 5;$$

$$p \approx \hat{p} \pm Z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} = \hat{p} \pm ME$$

Donde:

- $\hat{p}$ es la proporción muestral (estimador de la proporción poblacional p),
- $Z_{\alpha/2}$ es el valor crítico de la distribución normal,
- ME representa la distancia desde la estimación de la muestra hasta los puntos finales del intervalo de confianza E .

De aquí se deduce que el tamaño muestral debe ser siempre $n \geq 10$, ya que para que el estimador funcione deben existir al menos 5 éxitos y 5 fracasos:

$$\begin{aligned} n\hat{p} &\geq 5 \\ n \cdot \left(\frac{x}{n}\right) &\geq 5 \\ x &\geq 5 \end{aligned}$$

$$\begin{aligned} n(1 - \hat{p}) &\geq 5 \\ n \cdot \left(1 - \frac{x}{n}\right) &\geq 5 \\ n - x &\geq 5 \end{aligned}$$

Dentro de este marco, la métrica de *Accuracy* del modelo, calculada para un conjunto de datos de evaluación representativo, puede servir como una estimación puntual $\hat{p}$ de la verdadera proporción de predicciones correctas para la población del lote.

$$\hat{p} = \frac{\text{aciertos}}{n} \equiv \text{Accuracy}$$

Siempre y cuando se realice un número n de ensayos suficientemente grande, independientes entre sí, con distribución de Bernoulli, cada uno con probabilidad de éxito p , donde la distribución del número de éxitos se aproxima a una distribución normal.

Es importante destacar que para que dicha aproximación funcione debe tratarse de un modelo "imperfecto".

Si el modelo acierta todas las predicciones, la probabilidad de éxito será $p = 1$. De modo contrario, si nunca acierta, la probabilidad será $p = 0$. En ambos casos, dejan de cumplirse las condiciones necesarias para aplicar el Intervalo de Wald: $n\hat{p} \geq 5$ (éxitos) y $n(1 - \hat{p}) \geq 5$ (fracasos).

B.2. Impacto del tamaño muestral en el *Accuracy*

La proporción estimada $p(1 - p)$ representa la varianza de una variable aleatoria de Bernoulli con probabilidad de éxito p .

Si p es la verdadera proporción de una característica en una población, entonces $p(1 - p)$ representará la varianza de una observación individual de Bernoulli.

En estadística, la varianza de una proporción muestral $\hat{p}$ (estimada desde una población con proporción verdadera p) se aproxima teóricamente a $p(1 - p)/n$, siendo n el número de muestras.

En el contexto del *Machine Learning*, si bien puede considerarse al *Accuracy* como una estimación de la verdadera probabilidad de que el modelo clasifique acertadamente, $\hat{p} = \text{Accuracy}$, no es común utilizar directamente la expresión $\text{Accuracy}(1 - \text{Accuracy})$ como métrica de rendimiento, ya que esta alcanza su mínimo para valores de $\text{Accuracy} = 0$ o $\text{Accuracy} = 1$, y su máximo para $\text{Accuracy} = 0,5$.

Por este motivo, no se relaciona adecuadamente con la idea de mejora de la métrica. En cambio, funciona como una medida de la variabilidad intrínseca propia de esta métrica calculada sobre una muestra finita.

En su lugar, es más apropiado utilizar la métrica Precisión como proporción estimada de éxito, ya que describe efectivamente la proporción entre aciertos y errores en la detección. Otra opción, cuando no se cuenta con información previa, es utilizar un valor de $p = 0,5$ como estimación conservadora.

Tamaño muestral necesario para inferencia

La fórmula para calcular el tamaño de muestra necesario (n) para estimar una proporción poblacional (p) con un margen de error (E) y un nivel de confianza (Z) es:

$$n = \frac{Z^2 \cdot p \cdot (1 - p)}{E^2}$$

Donde:

- n es el tamaño de muestra necesario,
- Z es el valor crítico correspondiente al nivel de confianza deseado (p.ej. $Z = 1,96$ para 95 % de confianza, $Z \approx 1,28$ para 80 % de confianza),
- p es la proporción estimada de éxito (en este caso, la Precisión del modelo),
- E es el margen de error permitido (en este caso, 0,05 para $\pm 5\%$).

Luego, sabiendo que el área cubierta por la imagen aérea es $\sim 10\text{ m}^2$, puede realizarse un ajuste para poblaciones finitas:

$$n_{\text{ajustado}} = \frac{n}{1 + \frac{n-1}{M}}$$

Donde M es el número total de posibles *clusters* (fotos) en el lote:

$$M = \frac{\text{Área total del lote}}{\text{Área cubierta por foto}} = \frac{2\,000\,000\text{ m}^2}{10\text{ m}^2/\text{foto}} = 200\,000\text{ clusters (fotos)}$$

Caso 1

Por las pruebas realizadas con el modelo, se cuenta con la métrica Precisión, por lo que usaremos este parámetro como valor p :

$$p = \text{Precision} = 0,78 \quad (1 - p) = 1 - 0,78 = 0,22$$

Para el nivel de confianza calculado (de $\approx 80\%$), se utiliza $Z = 0,896$:

$$\begin{aligned} n_{80\% \text{ approx}} &= \frac{Z^2 \cdot p \cdot (1 - p)}{E^2} = \frac{(0,896)^2 \cdot 0,78 \cdot 0,22}{0,05^2} = \\ &\approx \frac{0,802816 \cdot 0,1716}{0,0025} \approx \frac{0,137763}{0,0025} \approx 55,10 \end{aligned}$$

$$\begin{aligned} n_{\text{ajustado}} &= \frac{n}{1 + \frac{n-1}{M}} = \\ &= \frac{55,10}{1 + \frac{55,10-1}{200\,000}} = \frac{55,10}{1,00027053} \approx 55,085 \end{aligned}$$

Luego, redondeando al entero más próximo, se puede concluir que el número de muestras necesarias para obtener una medición estadísticamente representativa, dada la precisión del modelo actual, es de 55 fotos para una confianza del 80 % con error del 5 % (valores aceptables para el cliente).

Caso 2

De modo inverso, podemos calcular la precisión p que debe alcanzar el modelo si se desea trabajar con una restricción del número de muestras $n = 30$, manteniendo $E = 0,05$ y $Z = 0,896$ (para $\sim 80\%$ de confianza):

$$n = \frac{Z^2 \cdot p \cdot (1 - p)}{E^2}$$

$$nE^2 = Z^2 \cdot p \cdot (1 - p)$$

$$p \cdot (1 - p) = \frac{nE^2}{Z^2}$$

Sustituyendo los valores, para $n = 30$:

$$p_{30} \cdot (1 - p_{30}) = \frac{30 \cdot (0,05)^2}{(0,896)^2} = \frac{30 \cdot 0,0025}{0,802816} \approx \frac{0,075}{0,802816} \approx 0,09342116$$

$$p_{30}^2 - p_{30} + 0,09342116 = 0$$

Resolviendo la ecuación cuadrática de la forma $ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$, donde $a = 1$, $b = -1$, $c = 0,09342116$:

$$\begin{aligned} p_{30} &= \frac{1 \pm \sqrt{(-1)^2 - 4 \cdot 1 \cdot 0,09342116}}{2 \cdot 1} = \frac{1 \pm \sqrt{1 - 0,37368464}}{2} = \\ &= \frac{1 \pm \sqrt{0,62631536}}{2} \approx \frac{1 \pm 0,79140088}{2} \end{aligned}$$

$$\begin{cases} p_{30A} = \frac{1+0,79140088}{2} \approx 0,89570044 \\ p_{30B} = \frac{1-0,79140088}{2} \approx 0,10429956 \end{cases}$$

Podemos ver que, de los valores que resuelven la ecuación cuadrática, uno (p_{30B}) no tiene sentido práctico si se busca mejorar la precisión del modelo (a menos que el modelo actual sea peor que aleatorio y se quiera una precisión muy baja pero específica). Generalmente se busca la mayor precisión.

$$Precision_{30} \approx 89,57$$

En conclusión, para poder trabajar solo con 30 imágenes y exigencias similares a las actuales del cliente (Nivel de Confianza $\sim 80\%$ con $Z = 0,896$ y Margen de Error 5%), el modelo debería alcanzar una precisión de $\sim 90\%$.

Apéndice C

Protocolo de vuelo de dron

Objetivo

Captura de imágenes en cultivos de arroz postemergencia en etapa temprana de crecimiento para desarrollar modelos de inteligencia artificial que faciliten y agilicen el conteo de plantas.

Requerimientos (Lista de Control)

- ☐ Incluir panel de reflectancia en la primera imagen.
- ☐ Verificar configuración de vuelo.
- ☐ Ajustar resolución y verificar el brillo.
- ☐ Verificar condiciones de vuelo favorables (viento).
- ☐ Al finalizar, verificar que las imágenes estén enfocadas.

Planes de Vuelo

Modelos de Dron

DJI Mavic 2 Pro, DJI Mavic Air 2 o DJI Mavic 3T.

Vuelo #1 - Captura Cenital

- Altura de vuelo: 3 metros.
- Ángulo de captura: 90° (cenital).
- Número de paradas por lote: 10.
- Resolución de imagen: Máxima, 48MP (Air2 o 3T) o 20 MP (2Pro).
- Modo de captura: Single shot.
- Modalidad de medición: Promedio ponderado al centro.
- Programa de exposición: Normal.
- Formato de archivo: JPEG.

Vuelo #2 - Captura Angulada

- Altura de vuelo: 3 metros.
- Ángulo de captura: 60°.
- Número de paradas por lote: 10.
- Resolución de imagen: Máxima, 48MP (Air2 o 3T) o 20 MP (2Pro).
- Modo de captura: Single shot.
- Modalidad de medición: Promedio ponderado al centro.
- Programa de exposición: Normal.
- Formato de archivo: JPEG.

Apéndice D

Recursos de hardware

Para el entrenamiento de modelos se utilizaron recursos de hardware ofrecidos por Google Colab. En particular, se trabajó con la tarjeta gráfica NVIDIA Tesla T4, cuyas especificaciones pueden ser consultadas en la tabla [D.1](#).

TABLA D.1. Especificaciones de la GPU NVIDIA Tesla T4.

Característica	Valor
Arquitectura	NVIDIA Turing
CUDA Cores	2,560
Tensor Cores	320
Memoria	16 GB GDDR6
Ancho de banda de memoria	Hasta 320 GB/s
Rendimiento FP32 pico	8.1 TFLOPS
Precisión mixta (FP16/FP32)	65 FP16 TFLOPS
Rendimiento INT8	130 INT8 TOPS
Rendimiento INT4	260 INT4 TOPS
Límite de potencia	70W
Formato	Single-slot, low-profile, 6.6-inch
Enfriamiento	Pasivo, requiere flujo de aire del sistema
Interfaz del sistema	PCIe 3.0 x16

Bibliografía

- [1] Adecoagro S.A. *1Q24 Earnings Release*. www.adecoagro.com. 2024. (Visitado 17-03-2025).
- [2] Adecoagro S.A. *Arroz*. Disponible: 2025-03-25. URL: <https://www.adecoagro.com/es/nuestros-negocios/arroz>.
- [3] Adecoagro. *Relación con Inversores*. <https://ir.adecoagro.com/result-center/>. Informes trimestrales entre 1Q20 a 1Q24. (Visitado 17-03-2025).
- [4] H. García-Martínez et al. «Digital Count of Corn Plants Using Images Taken by Unmanned Aerial Vehicles and Cross Correlation of Templates». En: *Remote Sensing* 12.7 (2020), pág. 1101. DOI: [10.3390/rs12071101](https://doi.org/10.3390/rs12071101).
- [5] J. Valente et al. «Automated crop plant counting from very high-resolution aerial imagery». En: *Precision Agriculture* 21.1 (2020), págs. 1366-1384. DOI: [10.1007/s11119-020-09725-3](https://doi.org/10.1007/s11119-020-09725-3).
- [6] S. Madec et al. «Ear density estimation from high resolution RGB imagery using deep learning technique». En: *Agricultural and Forest Meteorology* 264 (2019), págs. 225-234. DOI: [10.1016/j.agrformet.2018.10.013](https://doi.org/10.1016/j.agrformet.2018.10.013).
- [7] X. Xu et al. «Detection and counting of maize leaves based on two-stage deep learning with UAV-based RGB image». En: *Remote Sensing* 14.21 (2022), pág. 5388. DOI: [10.3390/rs14215388](https://doi.org/10.3390/rs14215388).
- [8] A. Ammar, A. Koubaa y B. Benjdira. «Deep-Learning-Based Automated Palm Tree Counting and Geolocation in Large Farms from Aerial Geotagged Images». En: *Agronomy* 11.8 (2021), pág. 1458. DOI: [10.3390/agronomy11081458](https://doi.org/10.3390/agronomy11081458).
- [9] B. Wang et al. «Multiscale Maize Tassel Identification Based on Improved RetinaNet Model and UAV Images». En: *Remote Sensing* 15.10 (2023), pág. 2530. DOI: [10.3390/rs15102530](https://doi.org/10.3390/rs15102530).
- [10] A. Karami, M. Crawford y E. J. Delp. «Automatic Plant Counting and Location Based on a Few-Shot Learning Technique». En: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 13.1 (2020), págs. 5872-5886. DOI: [10.1109/JSTARS.2020.3025790](https://doi.org/10.1109/JSTARS.2020.3025790).
- [11] L. Wang et al. «A Convolutional Neural Network-Based Method for Corn Stand Counting in the Field». En: *Sensors* 21.2 (2021), pág. 507. DOI: [10.3390/s21020507](https://doi.org/10.3390/s21020507).
- [12] X. Luo et al. «Fast Automatic Vehicle Detection in UAV Images Using Convolutional Neural Networks». En: *Remote Sensing* 12.12 (2020), pág. 1994. DOI: [10.3390/rs12121994](https://doi.org/10.3390/rs12121994).
- [13] N. B. Tatini et al. «Yolov4 Based Rice Fields Classification from High-Resolution Images Taken by Drones». En: *Proceedings of the IGARSS 2022-2022 IEEE International Geoscience and Remote Sensing Symposium*. IEEE. 2022, págs. 5043-5046.
- [14] R. Yang et al. «Lightweight Small Target Detection Algorithm with Multi-Feature Fusion». En: *Electronics* 12.12 (2023), pág. 2739. DOI: [10.3390/electronics12122739](https://doi.org/10.3390/electronics12122739).
- [15] X. Luo, Y. Wu y F. Wang. «Target Detection Method of UAV Aerial Imagery Based on Improved YOLOv5». En: *Remote Sensing* 14.19 (2022), pág. 5063. DOI: [10.3390/rs14195063](https://doi.org/10.3390/rs14195063).

- [16] J.-F. Yeh et al. «Automatic Counting and Location Labeling of Rice Seedlings from Unmanned Aerial Vehicle Images». En: *Electronics* 13.1 (2024), pág. 273. DOI: [10.3390/electronics13020273](https://doi.org/10.3390/electronics13020273).
- [17] M. Yao et al. «Rice Counting and Localization in Unmanned Aerial Vehicle Imagery Using Enhanced Feature Fusion». En: *Agronomy* 14.1 (2024), pág. 868. DOI: [10.3390/agronomy14040868](https://doi.org/10.3390/agronomy14040868).
- [18] DJI. *Mavic 2 User Manual*. Versión del manual de usuario con especificaciones técnicas. DJI Technology Co. y Ltd. 2018. URL: <https://www.dji.com/mavic-2/info#downloads>.
- [19] Roboflow. *YOLOv8 PyTorch TXT Format*. <https://roboflow.com/formats/yolov8-pytorch-txt>. Disponible: 2025-03-25.
- [20] IBM Cloud Education. *Aumento de datos: técnicas y ejemplos*. Disponible: 2025-03-25. URL: <https://www.ibm.com/es-es/think/topics/data-augmentation> (visitado 27-10-2023).
- [21] Ultralytics. *Data Augmentation*. Disponible: 2025-03-25. URL: <https://www.ultralytics.com/glossary/data-augmentation>.
- [22] Mohammad Mustafa Taye. «Theoretical Understanding of Convolutional Neural Network: Concepts, Architectures, Applications, Future Directions». En: *Computation* 11.3 (2023), pág. 52. DOI: [10.3390/computation11030052](https://doi.org/10.3390/computation11030052). URL: <https://doi.org/10.3390/computation11030052>.
- [23] Joseph Redmon et al. «You Only Look Once: Unified, Real-Time Object Detection». En: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, págs. 779-788. DOI: [10.1109/CVPR.2016.91](https://doi.org/10.1109/CVPR.2016.91).
- [24] Joseph Redmon y Ali Farhadi. «YOLOv3: An Incremental Improvement». En: *arXiv preprint arXiv:1804.02767* (2018). DOI: [10.48550/arXiv.1804.02767](https://doi.org/10.48550/arXiv.1804.02767). arXiv: [1804.02767](https://arxiv.org/abs/1804.02767) [cs.CV].
- [25] Ultralytics. *YOLOv8 Data Augmentation*. Disponible: 2025-03-25. URL: <https://yolov8.org/yolov8-data-augmentation/>.
- [26] Rejin Varghese y Sambath M. «YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness». En: *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*. IEEE, 2024. DOI: [10.1109/ADICS58448.2024.10533619](https://doi.org/10.1109/ADICS58448.2024.10533619).
- [27] Muhammad Yaseen. «What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector». En: *arXiv preprint arXiv:2408.15857* (2024). DOI: [10.48550/arXiv.2408.15857](https://doi.org/10.48550/arXiv.2408.15857). arXiv: [2408.15857](https://arxiv.org/abs/2408.15857) [cs.CV].
- [28] IBM Cloud Education. *Fine-tuning*. <https://www.ibm.com/think/topics/fine-tuning>. Disponible: 2025-03-25. n.d.
- [29] Martin Ester et al. «A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise». En: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*. Portland, Oregon, USA: AAAI Press, 1996, págs. 226-231.
- [30] scikit-learn developers. *sklearn.cluster.DBSCAN — scikit-learn 1.3.0 documentation*. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>. Disponible: 2025-03-25. 2023.
- [31] George Seif. *DBSCAN Clustering Algorithm in Machine Learning*. <https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html>. Disponible: 2025-03-25. 2020.
- [32] M. R. Golzarian, M.-K. Lee y J. M. A. Desbiolles. «Evaluation of Color Indices for Improved Segmentation of Plant Images». En: *Not specified in the*

- search results, likely *Computers and Electronics in Agriculture* based on other results (2012). URL: https://www.researchgate.net/publication/270607450_Evaluation_of_Color_Indices_for_Improved_Segmentation_of_Plant_Images.
- [33] D. M. Woebbecke et al. «Color indices for weed detection in row crops». En: *Transactions of the ASAE* 38.1 (1995), págs. 259-267.
- [34] G. E. Meyer y J. C. Neto. «Verification of color vegetation indices for automated crop imaging applications». En: *Computers and Electronics in Agriculture* 63.2 (2008), págs. 227-238.
- [35] Jiawei Han, Micheline Kamber y Jian Pei. *Data Mining: Concepts and Techniques*. 3rd. Morgan Kaufmann, 2012.
- [36] A. S. M. Anisur Rahman et al. «Dimension reduction of multispectral images using PCA and folded PCA». En: *ResearchGate* (2017). Highlights how PCA uses orthogonal transformation to convert high-dimensional data into linearly uncorrelated variables.
- [37] Karl Pearson. «On Lines and Planes of Closest Fit to Systems of Points in Space». En: *Philosophical Magazine* 2.11 (1901), págs. 559-572.
- [38] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.
- [39] Abinaya P y Sangeeta K. «Evaluation of PCA Based Image Denoising Methods». En: *International Journal of Applied Engineering Research* 10.17 (2015), págs. 38136-38145. URL: https://www.ripublication.com/ijaer10/ijaerv10n17_134.pdf.
- [40] Karel Zuiderveld. «Contrast Limited Adaptive Histogram Equalization». En: *Graphics Gems IV*. Ed. por Peter de Graaf, Max A. Viergever y Jan M. Thijssen. Academic Press, 1994, págs. 474-485.
- [41] Hyeon Kim, Seonghyun Lim y Hyungjoo Jeon. «Contrast Enhancement-Based Preprocessing Process to Improve Deep Learning Object Task Performance and Results». En: *Applied Sciences* 13 (19 2023), pág. 10760. DOI: 10.3390/app131910760. URL: <https://www.mdpi.com/2076-3417/13/19/10760>.
- [42] Alexander Buslaev et al. *Albumentations: Fast and Flexible Image Augmentations*. 2020. URL: <https://albumentations.ai/>.
- [43] F. Pedregosa et al. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830. URL: <http://jmlr.org/papers/v12/pedregosa11a.html>.
- [44] C. R. Harris et al. «Array programming with NumPy». En: *Nature* 585.7825 (2020), págs. 357-362. DOI: 10.1038/s41586-020-2649-2.
- [45] Ultralytics. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. 2023.
- [46] Departamento de Matemáticas, Universidad de Arizona (s.f.) *Tabla normal estándar*. <https://math.arizona.edu/~rsims/ma464/standardnormaltable.pdf>. (Visitado 03-05-2025).